UNIVERSIDADE LaSalle
Viver é evoluir.

NOVA GRADUAÇÃO UNIVERSIDADE LA SALLE

UNIDADE 3

Requisitos de SOFTWARE



INOVAÇÃO E TECNOLOGIA

unidade
3

Especificação de Requisitos não Funcionais, Verificação e Validação de Requisitos

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Aprender as formas de registrar requisitos não funcionais de software, verificar e validar se os requisitos identificados atendem as necessidades do negócio.

Editora
LaSalle

Parte 1

Requisitos não Funcionais de Software

O conteúdo deste livro
é disponibilizado
por SAGAH.

SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Requisitos não funcionais de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Conceituar qualidade em uso e qualidade de produto de *software*.
- Identificar características e subcaracterísticas dos modelos de qualidade da ISO/IEC 25000.
- Aplicar a norma ISO/IEC 25000 para a descoberta de requisitos não funcionais de *software*.

Introdução

É inquestionável a importância das atividades relacionadas à engenharia de requisitos para que o projeto de *software* seja bem-sucedido e o produto resultante atenda aos propósitos pelos quais ele foi desenvolvido. Negligenciar as atividades relacionadas à identificação e à análise dos requisitos pode introduzir riscos ao sucesso do projeto, levando a custos mais elevados para o desenvolvimento, perda de prazos e até mesmo a descontinuidade do projeto ou do produto.

Uma das primeiras atividades no ciclo de desenvolvimento é a descoberta do que deve ser construído, ou seja, que funcionalidades o produto de *software* deve conter para agregar valor ao usuário. Mas essa é apenas uma parte da história! Se não formos capazes de compreender como essas funcionalidades devem ser entregues, podemos igualmente falhar.

Estamos falando aqui sobre as expectativas do usuário que estão além das funcionalidades — estamos falando dos requisitos de qualidade. Embora muitas vezes esses sejam requisitos invisíveis, ou seja, aqueles que ficam implícitos na elicitação, eles geralmente impactam seriamente na aceitação do produto. Nenhum usuário está disposto a conviver com produtos difíceis de aprender a usar, complexos de operar, lentos ou com riscos de segurança. Sem falar em produtos cujo custo de manutenção passa a ser um pesadelo para a empresa, pois aspectos relacionados à

arquitetura e à manutenibilidade foram negligenciados. Os requisitos de qualidade são parte dos requisitos não funcionais do *software*.

Neste capítulo você vai estudar a diferença entre qualidade em uso e qualidade de produto. Vai ainda ler sobre as características da família de normas ISO/IEC 25000 e como essa norma pode ser aplicada para a descoberta e a documentação dos requisitos não funcionais.

1 Diferentes visões da qualidade

Requisitos não funcionais podem ser compreendidos de duas formas: podem ser atributos de qualidade que o produto de *software* em si deve ter ou podem ser restrições que o projeto ou o processo de desenvolvimento deve atender.

Vamos começar falando das restrições do projeto, ou seja, os **requisitos não funcionais de projeto**. Entendemos por projeto "[...] um esforço temporário empreendido para produzir um produto, serviço ou resultado único" (PROJECT MANAGEMENT INSTITUTE, 2017, p. 720). Requisitos não funcionais relacionados ao projeto são aqueles que podem afetar o planejamento e a execução do projeto, como prazos, custos, recursos. São parâmetros geralmente impostos ao projeto e sobre os quais a equipe não tem gestão. Essa imposição pode ser advinda do cliente, da própria empresa desenvolvedora ou de legislações e regulamentações. Geralmente elas são provenientes do meio externo e são difíceis, se não impossíveis, de negociar.



Exemplo

Veja aqui alguns exemplos de requisitos não funcionais (restrições) de **projeto**:

- O sistema deverá ser entregue até o mês de agosto, quando começam as liquidações de inverno (restrição de prazo).
- O *software* deverá ser desenvolvido considerando a disponibilização de um analista de negócios, um desenvolvedor e um analista de testes (restrição de recursos).
- O desenvolvimento do sistema não poderá custar mais do que R$ 100.000,00 (restrição de custo).
- O arquiteto de *software* estará disponível de 01/03 a 31/03; depois disso, estará alocado a outro projeto (restrição de recurso).

É importante destacar que os requisitos de projeto podem vir a impactar na qualidade do produto desenvolvido. Se um prazo, por exemplo, for arbitrado sem levar em consideração as questões técnicas, mostrando-se insuficiente, é possível que atividades do ciclo sejam suprimidas, como a elicitação mais cuidadosa dos requisitos ou mesmo a eliminação da etapa inteira de testes.

Outro tipo comum de requisito não funcional é o **requisito não funcional de processo**, também chamado de **restrição de processo**. Um processo é a forma pela qual realizamos uma atividade; ele é constituído pelas tarefas, artefatos, papéis e ferramentas. Requisitos não funcionais de processo impõem restrições à forma como o produto é desenvolvido, ou seja, restrições de tecnologia, e à forma como a equipe se organiza e realiza o seu trabalho.

Na categoria dos requisitos não funcionais de processo, podemos ter padrões organizacionais que precisam ser aplicados. Por exemplo, no caso do governo federal, todos os *sites* que trabalham com governo eletrônico devem seguir o padrão e-PWG. Encaixam-se aqui também os padrões de marca que algumas empresas têm, que definem detalhes de como a marca e as cores da empresa são usadas em seus produtos.



Exemplo

Veja aqui alguns exemplos de requisitos não funcionais (restrições) de **processo**:

- A equipe deverá utilizar o método ágil SCRUM, com entregas a cada 4 semanas.
- Os padrões para integração com o sistema de pagamentos deverão ser seguidos.
- O projeto deverá ser desenvolvido em Java.
- O gerente de projetos deverá enviar relatórios semanais de acompanhamento ao cliente.

Embora requisitos de processo possam aparentemente não conter qualquer relação com a qualidade do produto, é fácil perceber que isso não é verdade. Se houver, por exemplo, uma necessidade de executar algoritmos pesados e que requerem muito poder de processamento e houver uma restrição de que os equipamentos atuais devam ser mantidos, pode ser que o desempenho do produto seja prejudicado se essas máquinas se mostrarem insuficientes.

Finalmente, temos os **requisitos não funcionais de produto**, que são geralmente associados a atributos de qualidade. Costumamos dizer que os requisitos funcionais dizem **o que** o sistema deve fazer e os requisitos não funcionais dizem **como** o sistema deve fazer.

Geralmente os requisitos não funcionais de produto são mais perceptíveis pelo usuário no momento em que ele executa o produto — é o que chamamos de **qualidade em uso**. No entanto, nem sempre o usuário especifica esses requisitos no momento da elicitação. Muitas vezes ele entende que são atributos óbvios que qualquer *software* deveria ter, como bom tempo de resposta ou facilidade de uso. Por considerá-los óbvios, muitas vezes ele não acha que precisaria mencioná-los. E é aí que começa o problema.

Nem todos os requisitos não funcionais serão facilmente adicionados ao produto depois que ele estiver pronto. Requisitos que são afetados pela arquitetura da aplicação, por exemplo, precisam ser identificados o mais cedo possível, antes que a definição da arquitetura tenha sido realizada.

É papel do analista de requisitos elicitar todos os tipos de requisitos, o que implica em focar a sua atenção na elicitação dos requisitos funcionais, mas também nos não funcionais. Cabe ao analista de requisitos compreender o contexto no qual o produto se insere, identificar as fontes de informação e elicitar todos os requisitos necessários, que incluem os requisitos funcionais, geralmente mais óbvios, e os requisitos não funcionais, geralmente menos explícitos, conforme mencionamos.

As mesmas técnicas utilizadas para elicitar os requisitos funcionais se aplicam também para os não funcionais, ou seja, podemos utilizar:

- entrevista;
- reunião;
- *brainstorming*;
- observação;
- questionário;
- análise do sistema anterior;
- análise de normas e regulamentações;
- análise de padrões aplicáveis ao segmento de negócio.

Embora a interação do analista de requisitos com os *stakeholders* em busca dos requisitos não funcionais de produto possa ocorrer de forma *ad hoc*, é possível realizar essa atividade tendo em vista um *framework* de apoio. Isso permite que o analista de requisitos desperte a atenção dos *stakeholders* para questões que podem, de outra forma, permanecer implícitas e, por isso mesmo, não tratadas durante o desenvolvimento.

Um desses *frameworks* é o conjunto de normas internacionais da ISO/IEC denominado SQuaRE —*Software and Systems Quality Requirements and Evaluation*, ou seja, Requisitos e Avaliação de Qualidade de *Software* e Sistemas (ISO/IEC, 2014).



Fique atento

Embora no nosso dia a dia utilizemos as expressões *software* e sistemas como se fossem sinônimos, é importante relembrar a sua distinção. De acordo com a norma internacional ISO/IEC/IEEE 12207:

- **Produto de *software*:** "[...] conjunto de programas de computador, procedimentos e possíveis documentos e dados associados" (ISO/IEC/IEEE, 2017, p. 8).
- **Sistema de *software*:** "[...] sistema para o qual o *software* é de importância primordial para os *stakeholders*. No caso mais geral, um sistema de *software* é composto por *software*, *hardware*, pessoas e procedimentos manuais" (ISO/IEC/IEEE, 2017, p. 8).

SQuaRE

A ISO (International Organization for Standardization) e a IEC (Internacional Electrotechnical Commission) constituem um sistema para a normatização em âmbito mundial. Juntas, elas têm trabalhado na concepção de normas internacionais em diversas áreas. Na área de tecnologia da informação, ISO e IEC estabeleceram o JTC1 (Joint Technical Committee 1), sob o qual existem subcomitês que tratam de assuntos específicos. O Subcomitê SC7 trata das questões relacionadas com a engenharia de *software* e sistemas.

No âmbito da qualidade de produto de *software*, o SC7 produziu o conjunto de normas denominado SQuaRE, cuja composição está representada na Figura 1. É comum essas normas serem chamadas de família 25000.

A ideia é a seguinte: cada uma das divisões engloba um conjunto de normas com a finalidade específica daquela divisão. A divisão **2500n**, no alto da Figura 1, compreende as normas que estabelecem o **modelo geral** do SQuaRE, bem como tudo que é comum às demais normas da família, como a definição de termos utilizados. Fazem parte dessa divisão as normas ISO/IEC 25000 e ISO/IEC 25001.



Figura 1. Família de normas ISO/IEC 25.000 — SQuaRE.

Fonte: Adaptada de ISO/IEC (2014).

A divisão **2501n** compreende as normas que apresentam os **modelos de qualidade** para produtos de *software* e sistemas, que são ISO/IEC 25010 (Modelo de Qualidade de Produto e Modelo de Qualidade em Uso) e ISO/IEC 25012 (Modelo de Qualidade de Dados).

A divisão **2502n** trata das questões relacionadas à **medição da qualidade**, como a definição matemática das medidas que podem ser usadas e como aplicá-las. Fazem parte desse grupo as normas ISO/IEC 25020, ISO/IEC 25021, ISO/IEC 25022, ISO/IEC 25023 e ISO/IEC 25024.

A divisão **2503n** trata das questões relacionadas aos **requisitos de qualidade**, abrangendo os processos de elicitação e avaliação. Compreende a norma ISO/IEC 25030.

A divisão **2504n** compreende as normas relacionadas à **avaliação da qualidade** de produto, quando aplicada por terceira parte independente. Compreende as normas ISO/IEC 25040, ISO/IEC 25041 e ISO/IEC 25045.

O intervalo de normas que vai desde a 2505n até a 2509n está reservado para normas e relatórios técnicos para domínios específicos de aplicação. Alguns deles já foram desenvolvidos pelo SC7 e se encontram disponíveis para compra no *site* da ISO, outros estão em fase de elaboração.

A série iniciada por 2506n é dedicada a questões relacionadas à usabilidade dos produtos de *software*. Nessa categoria já foram desenvolvidas as seguintes normas internacionais:

- ISO/IEC 25060 — *framework* geral para informações relacionadas à usabilidade.
- ISO/IEC 25062 — relatórios de teste de usabilidade.
- ISO/IEC 25063 — descrição do contexto de uso.
- ISO/IEC 25064 — relatório de necessidades do usuário.
- ISO/IEC 25065 — relatório de avaliação.



Saiba mais

Você sabe como as normas da ISO são criadas? A ISO é uma organização sem fins lucrativos e conta com comitês que são formados por entidades representativas dos diversos países a ela associados. Essas entidades, por sua vez, são compostas por pessoas provenientes do governo, da indústria e da academia. São essas pessoas que representam os interesses de cada país na confecção das normas internacionais. Essas entidades têm direito a propor, opinar, participar da elaboração e votar cada norma.

Aqui no Brasil, a ABNT (Associação Brasileira de Normas Técnicas) é o órgão oficial de normalização e é afiliada à ISO. Profissionais brasileiros participam voluntariamente dos diversos comitês da ABNT, que trabalham para contribuir para a confecção das normas internacionais e para a tradução delas para o português, quando estão aprovadas. Periodicamente a ABNT coloca os projetos de normas em consulta pública em seu *site* e todos podem contribuir com sugestões.

Para conhecer a história da ISO, busque na *web* pelo *site* oficial da ISO. É bem fácil de encontrar. Além de encontrar a história dessa importante instituição que tanto tem contribuído para a normalização no mundo, lá você também pode adquirir as normas internacionais em suas versões originais, em inglês.

Para saber mais sobre a ABNT e o processo de normalização brasileiro, procure na *web* pelo *site* oficial da entidade. Lá é possível adquirir as normas que já estão disponíveis em português e conhecer um pouco mais sobre como você também pode contribuir.

Modelos de qualidade na família ISO/IEC 25000

A norma internacional ISO/IEC 25010 (ISO/IEC, 2011) especifica os seguintes modelos de qualidade:

- Modelo de Qualidade em Uso.
- Modelo de Qualidade de Produto.

A norma internacional ISO/IEC 25012 (ISO/IEC, 2008) especifica o seguinte modelo de qualidade:

- Modelo de Qualidade de Dados.

Qualidade em uso significa o grau que determinado produto ou *software* pode ser usado por seus usuários para atender às suas necessidades com efetividade, eficiência, satisfação e riscos reduzidos, em determinados contextos de uso (ISO/IEC, 2011).

Entende-se que o usuário, ao utilizar o produto no ambiente-alvo, vivencia uma experiência que depende dessas características que, em última instância, lhes causam satisfação com o uso, ou não. Se um produto permite que o usuário desempenhe suas funções, no âmbito profissional ou pessoal, de forma eficiente, então ele estará satisfeito. Se, por outro lado, o *software* não se comportar da forma esperada, dificultando a realização das suas atividades, ele estará frustrado e insatisfeito com o produto. Se ele puder escolher, abandonará o seu uso e muito provavelmente se tornará um detrator do produto e da marca.

A qualidade em uso pode ser avaliada em dois momentos. O primeiro, ao longo do ciclo de desenvolvimento, como uma avaliação formativa, ou seja, aquela que contribui para as atividades do desenvolvimento. Ao término, quando o produto está efetivamente no ambiente-alvo, então se realiza a avaliação somativa dos atributos da qualidade em uso (ISO/IEC, 2016).

Qualidade de produto analisa as propriedades relacionadas com adequação funcional, eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade (ISO/IEC, 2011). Esse conjunto de características é mais abrangente e pode ser avaliado para identificar a qualidade interna do produto e a qualidade externa.

Qualidade interna diz respeito às características do produto de *software* que não são observadas externamente pelo usuário, mas que interessam às equipes de desenvolvimento e manutenção de *software*. Por exemplo, um *software* bem estruturado pode facilitar a atividade de manutenção, seja para encontrar o local onde um *bug* foi originado, seja para identificar o ponto do produto onde uma melhoria será aplicada. Tipicamente esses requisitos não serão solicitados pelo usuário, mas pelos *stakeholders* internos, a menos que a manutenção seja dada pelo cliente, que, nesse caso, pode solicitar requisitos que afetam a qualidade interna.

A qualidade interna pode avaliar produtos intermediários do desenvolvimento de *software*, como o documento de requisitos, por exemplo. Isso pode ser feito antes que o sistema esteja pronto. O ideal é quando medidas de qualidade interna têm alguma forma de correlação com as medidas da qualidade externa, pois, nesses casos, uma pode ser utilizada para predizer a outra.

Qualidade externa diz respeito às características do produto pronto, que podem ser avaliadas antes que ele seja disponibilizado no ambiente-alvo. Nesse caso, são avaliadas em ambientes como desenvolvimento, teste e homologação, mas ainda não no ambiente real de produção.

Qualidade de dados foca, conforme o próprio nome sugere, na qualidade dos dados. O modelo de qualidade de dados define as características de qualidade dos dados como parte de um sistema de computador utilizado por humanos e sistemas (ISO/IEC, 2015). A qualidade de dados não será objeto de nossa análise neste capítulo.



Fique atento

Não é necessário que você saiba de cor as normas e muito menos seus números. A ideia é que você, ao elicitar requisitos, **esteja atento para os requisitos não funcionais**. Conhecer as normas vai ajudá-lo a elaborar *checklist* de características que podem se aplicar a cada contexto diferente de desenvolvimento, ajudando a tornar explícitos os requisitos implícitos.

É responsabilidade do analista de requisitos envidar todos os esforços para garantir que as necessidades dos *stakeholders* sejam explicitadas e adequadamente compreendidas. Requisitos não identificados, ou seja, aqueles que são invisíveis no primeiro momento, são as maiores causas de insatisfação com produtos de *software*. Isso independe se a equipe utiliza métodos ágeis ou abordagens mais tradicionais de desenvolvimento.

Os requisitos de qualidade podem, portanto, ser classificados com (ISO/IEC, 2019):

- requisitos de qualidade de *software* em uso;
- requisitos de qualidade de *software* externa;
- requisitos de qualidade de *software* interna.

2 Características e subcaracterísticas

Os modelos de qualidade da norma internacional ISO/IEC 25000 são estruturados no formato de características e subcaracterísticas, conforme será apresentado nesta seção. O objetivo é a organização dos elementos em uma estrutura que facilita o entendimento e a utilização.

Como pode ser observado na Figura 2, um modelo de qualidade é composto por características de qualidade que, por sua vez, podem ser decompostas em subcaracterísticas. Em alguns casos raros, essas subcaracterísticas podem ainda ser desmembradas em subsubcaracterísticas. Ao final, um conjunto de propriedades de qualidade é definido, permitindo que a qualidade se torne mensurável. Nem todas as características e subcaracterísticas que constam na norma internacional serão aplicáveis a todos os contextos e *stakeholders*.

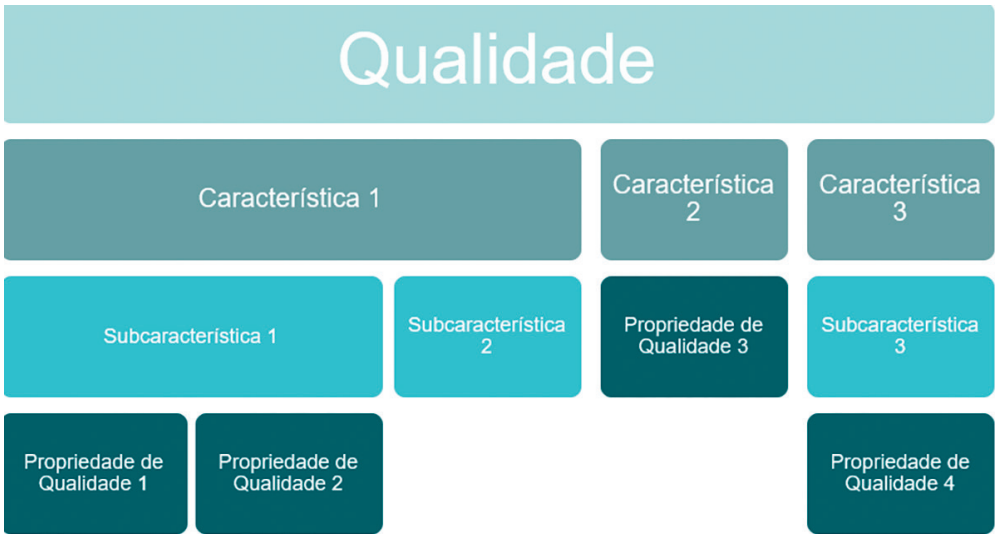


Figura 2. Estrutura dos modelos de qualidade da ISO/IEC 25010.

Fonte: Adaptada de ISO/IEC (2011).

Modelo de Qualidade de Produto

O Modelo de Qualidade de Produto é composto por 8 características e 31 subcaracterísticas. Nem todas elas farão sentido em todos os contextos, e cabe ao analista elicitar junto aos *stakeholders* aquelas que sejam mais adequadas e necessárias para o produto que está sendo desenvolvido. É comum que as pessoas se referenciem a essas características como as "*ilities*", devido ao sufixo da maior parte das palavras em inglês (*suitability, compatibility, usability* etc.). Em português, a maior parte acaba sendo traduzida para as "ades" (compatibilidade, usabilidade, portabilidade etc.).

A norma não pretende ser exaustiva nas características identificadas, mas entende-se que elas devem cobrir a maior parte das situações práticas. O Modelo de Qualidade de Produto é apresentado na Figura 3 e o detalhamento dos elementos, apresentaremos a seguir.



Figura 3. Modelo de Qualidade de Produto da ISO/IEC 25010.

Fonte: Adaptada de ISO/IEC (2011).

Adequação funcional

- **Completude funcional:** o quanto as funções disponibilizadas no produto cobrem todas as tarefas e as necessidades especificadas.
- **Corretude funcional:** o quanto a função fornece resultados corretos e precisos.
- **Funcionalidade apropriada:** o quanto as funções facilitam as tarefas que o usuário realiza.

Eficiência no desempenho

- **Comportamento em relação ao tempo:** desempenho do *software* em relação ao tempo (tempo de resposta, *throughput*).
- **Utilização de recursos:** consumo de recursos realizado pelo *software* (CPU, memória, espaço de armazenamento etc.).
- **Capacidade:** limites máximos dos parâmetros do produto (número máximo de usuários, banda de rede, transações etc.).

Compatibilidade

- **Coexistência:** capacidade de conviver com outros produtos no meio ambiente, compartilhando recursos, sem impactar no desempenho dos outros produtos.
- **Interoperabilidade:** grau em que dois ou mais produtos trocam informações e utilizam as informações trocadas.

Usabilidade

- **Reconhecimento de adequação:** grau de reconhecimento do usuário de que o produto é adequado às suas necessidades.
- **Apreensibilidade:** facilidade de o usuário aprender a utilizar o produto.
- **Operacionalidade:** facilidade de o usuário operar e controlar o produto.
- **Proteção contra erros do usuário:** proteção que o sistema oferece para evitar erros do usuário.
- **Estética da interface com o usuário:** o produto oferece prazer e satisfação na interface com o usuário.
- **Acessibilidade:** variedade de público com características diferentes que conseguem usar o produto.

Confiabilidade

- **Maturidade:** o quanto o produto é confiável sob condições normais.
- **Disponibilidade:** o quanto o produto está disponível e acessível quando necessário.
- **Tolerância a falhas:** o quanto o produto, *software* ou componente opera como pretendido na presença de falhas de *hardware* ou *software*.
- **Recuperabilidade:** como o sistema se recupera após uma falha e restabelece suas condições normais de operação.

Segurança

- **Confidencialidade:** dados são acessados apenas por aqueles que devem ter acesso.
- **Integridade:** prevenção do acesso por pessoas não autorizadas a dados e programas.
- **Não repúdio:** comprovação da realização de eventos ou ações de modo que não possam ser negados posteriormente.
- **Responsabilização:** rastreamento das ações a uma única entidade.
- **Autenticidade:** comprovação da identidade de uma pessoa ou elemento.

Manutenibilidade

- **Modularidade:** modularização dos componentes de modo que uma modificação em um componente tenha impacto mínimo em outros componentes.
- **Reusabilidade:** reutilização de ativos (módulos, programas, modelos etc.) em mais do que um sistema.
- **Analisabilidade:** facilidade de encontrar pontos de falhas ou pontos onde deverá ser realizada uma modificação, de forma eficiente e efetiva.
- **Modificabilidade:** facilidade de promover modificações de forma eficiente e efetiva, sem inserir defeitos ou degradar a qualidade do produto.
- **Testabilidade:** facilidade de estabelecer critérios e testar esses critérios de forma eficiente e efetiva.

Portabilidade

- **Adaptabilidade:** adaptação eficiente e efetiva a outros ambientes de *hardware* ou *software*.
- **Instalabilidade:** facilidade de instalar e desinstalar o produto de forma eficiente e efetiva em um ambiente específico.
- **Capacidade de substituir:** facilidade que o produto tem de substituir outro no mesmo ambiente e com a mesma função.

Modelo de Qualidade em Uso

O Modelo de Qualidade em Uso definido na norma internacional ISO/IE 25010 (ISO/IEC, 2011) é constituído por 5 características e 9 subcaracterísticas, que são apresentadas na Figura 4. Novamente, ressaltamos que nem todas elas serão utilizadas para todos os contextos.



Figura 4. Modelo de Qualidade em Uso da ISO/IEC 25010.
Fonte: Adaptada de ISO/IEC (2011).

Efetividade

Precisão e completude com que os usuários concluem suas tarefas (não tem subcaracterísticas associadas).

Eficiência

Recursos gastos em relação à eficiência conseguida (não tem subcaracterísticas associadas).

Satisfação

- **Utilidade:** satisfação dos usuários com os resultados e os efeitos do uso do produto.
- **Confiança:** confiança de que o produto se comporta como esperado.
- **Prazer:** prazer ao satisfazer às suas necessidades pelo uso do produto.
- **Conforto:** satisfação com o conforto físico.

Livre de risco

- **Mitigação de riscos econômicos:** mitigação de riscos como situação financeira, reputação, operação eficiente etc.
- **Mitigação de riscos de vida e proteção:** mitigação dos riscos relacionados às pessoas no contexto de uso.
- **Mitigação do risco ambiental:** mitigação dos riscos de propriedade e do meio ambiente.

Cobertura de contexto

- **Completude de contexto:** efetividade, eficiência, livre de risco e satisfação em todos os contextos de uso especificados.
- **Flexibilidade:** efetividade, eficiência, livre de risco e satisfação em todos os contextos além dos inicialmente especificados.



Fique atento

Normalmente é mais fácil que consigamos definir requisitos de forma adequada quando eles descrevem funcionalidades. Estamos mais acostumados a pensar **no que** o sistema deve fazer e não em **como** o sistema deve fazer. No entanto, é importante lembrar que tanto os requisitos funcionais quanto os requisitos não funcionais devem ser não ambíguos, verificáveis, consistentes, rastreáveis e comunicados a todos os interessados, ou seja, pessoas que terão que desenvolvê-los e pessoas que irão utilizá-los. Isso quer dizer que precisamos identificar claramente como eles poderão ser verificados e validados.

3 Aplicação das normas ISO/IEC 25000

Conforme discutimos na seção anterior, a família de normas ISO/IEC 25000 pode ser utilizada como um ***checklist*** para apoiar as etapas de elicitação de requisitos. Nem todos os produtos de *software* terão necessidade de todos os elementos que vimos, mas esse *checklist* vai atuar como uma ferramenta de apoio para que nenhum dos itens seja esquecido ou negligenciado no momento da elicitação.

Outra aplicação das normas ISO/IEC 25000 se refere à **concepção de métricas** que podem ser utilizadas para verificar se determinado requisito de qualidade foi implementado corretamente. Por exemplo, um requisito de qualidade pode definir que o sistema deva ter a capacidade de processar todas as consultas que ele recebe. Esse é um requisito muito genérico e precisa de mais informações, tanto para ser implementado quanto para ser validado posteriormente. Ele poderia ser refinado, por exemplo, estabelecendo que o sistema deverá processar 95% das consultas recebidas em um tempo não superior a 2 segundos.

A ISO/IEC 25022 (ISO/IEC, 2016a) e a ISO/IEC 25023 (ISO/IEC, 2016b) sugerem uma forma padronizada para a definição das medidas que poderão ser coletadas e analisadas, para determinar se o requisito de qualidade foi ou não atendido, que é a seguinte:

- **Identificação da medida:** um identificador único que identificará a medida.
- **Nome:** nome da medida, por extenso.
- **Descrição:** informação provida pela medida.
- **Função de medição:** como a métrica será calculada (como os elementos da métrica são combinados para produzir a métrica).
- **Método:** fórmula para o cálculo da medida, a partir dos elementos da medida.

Na sequência vamos ver alguns exemplos de como definir uma medida e como ela poderá ser usada para especificar um requisito não funcional.

Satisfação do usuário

Para ficar mais claro como as medidas podem ser definidas, vamos tomar como exemplo uma medida de qualidade em uso, relacionada com a **satisfação do usuário** ao utilizar o produto. A satisfação se refere ao "[...] grau no qual as necessidades dos usuários são satisfeitas quando um produto ou sistema é usado em um contexto de uso especificado" (ISO/IEC, 2011, p. 8).

A satisfação é uma característica que está dividida em 4 subcaracterísticas: utilidade, confiança, prazer e conforto. Podemos ter uma medida única para medir a satisfação geral, ou definir medidas para cada uma das subcaracterísticas individualmente.

Existem diversas formas de medir a satisfação do usuário. Uma delas é por meio de uma pesquisa de opinião, usando questionário. Esse questionário pode ser bem simples, composto de uma única questão que pergunta sobre a satisfação, ou pode ser mais detalhado, buscando compreender pontos específicos de maior ou de menor satisfação. Independentemente da forma escolhida, é necessário definir medidas que possam servir de indicadores da satisfação.

O Quadro 1 traz um exemplo de como a medida de satisfação pode ser definida.

Quadro 1. Modelo de especificação de medidas — satisfação

ID	Nome	Descrição	Função de medição	Método
SUs-1-G	Satisfação geral	Satisfação geral do usuário	$X = \sum Ri$ onde: R = resposta a uma questão i	Questionário

Fonte: Adaptado de Norma Internacional ISO/IEC 25022 (2016a).

Este é um exemplo de uma especificação de medida que pode ser utilizada para medir a satisfação do usuário. Um dos requisitos ao iniciar um desenvolvimento de *software* é atingir determinado nível para essa satisfação, e é isto que deve ser especificado como requisito não funcional. Por exemplo, pode-se estabelecer que se deseja que a satisfação dos usuários tenha uma média 8,5, considerando notas de 0 a 10.



Saiba mais

Uma das formas de medir a satisfação do usuário que vem sendo muito utilizada, especialmente para medir qualidade de serviços, é utilizar o NPS (*net promoter score*).

Para conhecer mais sobre NPS, consulte o artigo original de Frederick Reichheld, publicado na *Harvard Business Review* em 2003, intitulado "The One Number You Need to Grow" (O único número que você precisa para crescer). Basicamente, ele avalia a probabilidade de um consumidor indicar o seu produto.

Efetividade

Efetividade é uma característica estabelecida pela ISO/IEC 25010 (ISO/IEC, 2011) que se refere à "[…] precisão e completude com as quais o usuário atinge seus objetivos específicos" (ISO/IEC, 2011, p. 8). Ela vai avaliar se os objetivos do usuário foram atingidos, e não como eles foram atingidos.

O Quadro 2 traz um exemplo de como uma medida poderia ser estabelecida para a efetividade. Para definir os requisitos relacionados a esse aspecto, poderia ser estabelecido um limite desejado, que poderia inclusive ser 100%. A forma de se avaliar posteriormente é colocar usuários típicos (como os que foram definidos com as personas) para testar o *software*. A partir daí, realizar a medida de quantas tarefas ele consegue completar com sucesso e sem auxílio.

Quadro 2. Modelo de especificação de medida — efetividade

ID	Nome	Descrição	Função de medição	Método
Ef-1-G	Tarefas completadas	Percentual de tarefas que são completadas com sucesso e sem assistência	$X = A/B$ onde: ■ *A* é a quantidade de tarefas completadas com sucesso ■ *B* é a quantidade total de tarefas pretendidas	Medir o desempenho do usuário

Fonte: Adaptado de Norma Internacional ISO/IEC 25022 (2016a).

Usabilidade

Uma das características mais perceptíveis pelo usuário final de um *software* que envolve interação direta com o usuário é a **usabilidade**. Produtos cuja interface dificulta o seu uso estão fadados ao fracasso e ao abandono. Isso é válido para os produtos de *software* interativos, ou seja, que apresentam uma interação direta com o usuário. Não é à toa que o comitê conjunto da ISO/IEC optou por investir em projetos de normas relacionadas à usabilidade, com a série 2506n (25060, 25062 etc.).

A área de usabilidade é uma área específica que se situa entre a engenharia de *software* e o *design*. Recentemente, as organizações têm dedicado especial atenção ao envolvimento dos profissionais de *design* logo no início do ciclo de vida de desenvolvimento do *software*, de modo que as questões de usabilidade possam ser tratadas desde o início. Na maioria das organizações, entretanto, esse trabalho é feito pelos desenvolvedores, no momento da implementação.

Independentemente de haver ou não um profissional que atue diretamente com as questões de usabilidade na organização, esses requisitos precisam ser identificados para que possam ser implementados pela equipe de desenvolvimento. O primeiro passo é identificar quais são as características do contexto onde o produto será utilizado; para isso podemos iniciar estabelecendo as personas. Mapear as personas significa delinear as características típicas de um grupo de usuários do sistema, permitindo que se identifique aspectos relevantes que afetam a usabilidade. Pessoas com grande acesso a produtos de *software* terão requisitos diferentes de pessoas com menos proximidade com a tecnologia. Quando mapeamos as personas e identificamos as classes de usuários, já estamos dando um passo importante para identificar requisitos de usabilidade.

Wiegers e Beatty (2013, p. 196) apontam alguns itens que devem ser levados em consideração quando você estiver projetando as interfaces com o usuário ou quando estiver identificando os requisitos não funcionais relacionados à usabilidade. Veja a seguir.

- Referências a padrões de interface ou guias de estilo para uma linha de produto que devem ser seguidos.
- Padrões para fontes, ícones, rótulos de botões, imagens, esquemas de cores, sequências de avanço dos campos (*tab*), controles usados comumente, padrão gráfico da marca, avisos de *copyright* e privacidade, e coisas do gênero.
- Tamanho da tela, *layout* ou restrições de resolução.
- Padrões para botões, funções ou *links* de navegação que vão aparecer em cada tela, como botões de ajuda.
- Teclas de atalho.
- Visualização de mensagens e convenções de escrita.
- Guias para validação de dados (como restrições de valores de entrada e quando validar o conteúdo de um campo).
- Padrões de *layout* para facilitar a localização no *software*.
- Acomodações para usuários com deficiência visual, daltonismo, ou que tenham outras limitações.

Conforme vimos na seção anterior, a usabilidade é uma característica que engloba as seguintes subcaracterísticas: reconhecimento de adequação; apreensibilidade; operacionalidade; proteção contra erros do usuário; estética da interface com o usuário; e acessibilidade.

Vamos ver o exemplo de uma medida para avaliar a apreensibilidade, ilustrado no Quadro 3.

Quadro 3. Modelo de especificação de medida — apreensibilidade

ID	Nome	Descrição	Função de medição	Método
APR-1	Compreensão das mensagens de erro	Percentual das mensagens de erro que estabelece a razão pela qual o erro ocorreu e como resolvê-lo	$X = A/B$ onde: ■ *A* é a quantidade de mensagens que estabelece a razão da ocorrência e sugere formas de resolver onde isso é possível ■ *B* é a quantidade total de mensagens implementadas	Medir as quantidades de mensagens

Fonte: Adaptado de Norma Internacional ISO/IEC 25023 (2016b).

Outras medidas

Diversas são as medidas que podem ser utilizadas para tornar os requisitos não funcionais mais específicos e passíveis de verificação ao final. Se os requisitos não funcionais não forem especificados em grau de detalhe suficiente, ao final do desenvolvimento pode ser que não haja consenso entre a equipe desenvolvedora e o cliente, se eles foram atingidos ou não.

As normas da ISO/IEC podem ajudar a estabelecer essas medidas, pois um conjunto abrangente delas é especificado nos seguintes documentos:

- ISO/IEC 25022 (ISO/IEC, 2016a) — Medição da qualidade em uso.
- ISO/IEC 25023 (ISO/IEC, 2016b) — Medição da qualidade de produto de sistemas e *software*.
- ISO/IEC 25024 (ISO/IEC, 2015) — Medição da qualidade de dados.



Referências

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION (ISO; IEC). *ISO/IEC 25000:2014*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), guide to SQuaRE. Switezland: ISO, 2014.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION (ISO; IEC). *ISO/IEC 25010:2011*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), system and software quality models. Switzerland: ISO, 2011.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION (ISO; IEC). *ISO/IEC 25012:2008*: software engineering, software product quality requirements and evaluation (SQuaRE), data quality model. Switzerland: ISO, 2008.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION (ISO; IEC). *ISO/IEC 25022:2016*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), measurement of quality in use. Switezland: ISO, 2016a.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION (ISO; IEC). *ISO/IEC 25023:2016*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), measurement of system and software product quality. Switezland: ISO, 2016b.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION (ISO; IEC). *ISO/IEC 25024:2015*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), measurement of data quality. Switezland: ISO, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION (ISO; IEC). *ISO/IEC 25030:2019*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), quality requirementes framework. Switezland: ISO, 2019.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION; IEEE STANDARDS ASSOCIATION (ISO; IEC; IEEE). *ISO/IEC/IEEE 12207:2017*: systems and software engineering, software life cycle processes. Switzerland: ISO, 2017.

PROJECT MANAGEMENT INSTITUTE. *Um guia do conhecimento em gerenciamento de projetos (guia PMBOK)*. 6. ed. Newton Square: Project Management Institute, 2017.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3rd ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

Parte 2

Especificação dos Requisitos Não Funcionais de Software

O conteúdo deste livro é disponibilizado por SAGAH.

SOLUÇÕES EDUCACIONAIS INTEGRADAS

Especificação dos requisitos não funcionais de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar os requisitos não funcionais de um projeto de *software*.
- Especificar os requisitos não funcionais de *software*.
- Selecionar indicadores para avaliar os requisitos não funcionais de *software*.

Introdução

Com a intensa proliferação de produtos tecnológicos no dia a dia das pessoas, as exigências do mercado ficaram mais altas. Todos somos rotineiramente expostos a diversos *softwares*, especialmente na forma de aplicativos nos *smartphones*. Com a mesma facilidade com que são baixados novos *apps*, eles também são deletados. Essa grande exposição a vários tipos de produto torna o usuário cada vez mais exigente, seja com a qualidade da interface, seja com a praticidade do uso, seja com a satisfação que as funcionalidades lhe proporcionam. Negligenciar esses requisitos de qualidade é deixar de lado um dos maiores fatores de conquista do usuário.

Requisitos não funcionais esquecidos ou não implementados adequadamente são uma das maiores causas de abandono de produtos de *software*. Há mais do que um conjunto de funcionalidades em um *software* de sucesso. Quando um *software* falha em entregar requisitos como bom desempenho, facilidade de uso, segurança e mais uma série de outros atributos de qualidade, é muito provável que ele tenha vida curta no mercado.

Quando determinados requisitos não funcionais são descobertos tardiamente no ciclo de desenvolvimento de *software*, pode não ser mais possível resolver o problema, ou pode ser muito caro resolvê-lo. Se decisões arquiteturais forem tomadas sem levar em consideração os requisitos não funcionais (de escalabilidade, por exemplo), pode ser que não seja possível implementar esse requisito sem que uma grande mudança seja feita. Isso pode inviabilizar a continuidade de um projeto ou de um produto de *software*, devido aos altos custos. Em casos extremos, pode causar até mesmo perda de vidas.

Neste capítulo você vai ver como identificar requisitos não funcionais de produtos de *software*. Vai também ler sobre como descrever esses requisitos e, por fim, sobre como definir indicadores que permitam sua verificação e validação.

1 Identificação de requisitos não funcionais de *software*

Certamente você já ouviu falar que requisitos não funcionais, também chamados de requisitos de qualidade, são mais difíceis de serem elicitados e especificados do que requisitos funcionais. Será que isso é verdade? Na maioria das vezes, sim. É comum que os *stakeholders* responsáveis por fornecer os requisitos não enxerguem, pelo menos à primeira vista, os requisitos não funcionais, e eles podem permanecer invisíveis por muito tempo.

Além disto, diferentes tipos de *software* terão diferentes tipos de requisitos não funcionais. *Softwares* com intensa interação com os usuários certamente terão requisitos distintos daqueles de *softwares* que estão embarcados em equipamentos. *Softwares* que desempenham funções críticas também terão demandas distintas de *softwares* que podem falhar. Dessa forma, o contexto de negócio e o tipo de aplicação irão influenciar sobremaneira os requisitos de qualidade aplicáveis.

Em sistemas interativos, ou seja, que são baseados no diálogo com o usuário, alguns requisitos não funcionais são fundamentais, como, por exemplo, requisitos relacionados à usabilidade. Usabilidade depende fortemente do contexto e do público-alvo. Se os usuários forem pessoas que têm intimidade com a tecnologia, é uma coisa; se os usuários forem pessoas leigas em tecnologia, é outra coisa bem diferente.

Se o *software* for disponibilizado sem treinamento, os requisitos de usabilidade aumentam. Por exemplo, *sites* que vendem produtos pela *web*. Não é esperado que as pessoas tenham que fazer um curso para aprender a usá-los, é esperado que elas acessem o *site* e consigam concluir sua compra com sucesso. O projeto da interface tem que ajudar o vendedor a não perder uma venda por problemas, por exemplo, de navegação no *site*.



Exemplo

Wiegers e Beatty (2013) citam o exemplo de um novo *software* com interface gráfica que foi instalado em uma equipe de *call center* em substituição ao sistema anterior, que era baseado em caracteres e teclas de atalho. Os usuários tinham uma grande produtividade no sistema anterior porque eram experientes e navegavam rapidamente com as teclas de atalho. Ao mudar para o novo sistema, o uso do *mouse*, intercalado com o uso do teclado, tirou-lhes a produtividade. Resumindo: o novo sistema teve que ser remodelado para contemplar a possibilidade da navegação com teclas de atalho.

Se estivermos falando de um sistema para o segmento bancário, teremos uma infinidade de requisitos não funcionais, mas certamente a segurança estará entre os de maior prioridade. Oferecer uma interface fácil de operar é importante também neste segmento, mas desde que os aspectos de segurança tenham sido adequadamente tratados.

Em sistemas de missão crítica relacionados a aspectos que envolvem vidas, por exemplo, haverá certamente requisitos não funcionais de os sistemas estarem livres de riscos. Este é o caso, por exemplo, de veículos autônomos. Uma operação incorreta pode custar a vida de pedestres ou de passageiros do próprio veículo. O mesmo ocorre para sistemas que podem oferecer riscos financeiros ou ambientais.

Existem os requisitos que não são diretamente visualizados pelos clientes e usuários, mas que são uma preocupação da equipe desenvolvedora — são os requisitos relacionados à qualidade interna do produto, como manutenibilidade, ou seja, facilidade de dar manutenção. Quantas vezes a equipe recebe um chamado relatando um *bug* que demora dias para ser localizado. Por que isto ocorre? Porque a arquitetura da aplicação não favorece a manutenção, dificultando a agilidade na solução de problemas ou na implementação de melhorias e evoluções.

Outro contexto específico é o das *startups* de tecnologia. É comum que elas comecem seu negócio baseadas em uma ideia inovadora dos sócios que se esforçam para colocar de pé um MVP (*minimum viable product* ou produto mínimo viável). Depois de um tempo, esse protótipo se transforma no produto que vai para o mercado. Como o tempo exigido pelo mercado é curto, a *startup* evolui o produto sem muito planejamento, afinal ela precisa escalar! Se os cuidados com a arquitetura e os requisitos de escalabilidade tiverem sido tomados, tudo certo; mas esse não é o cenário mais comum, e o produto evoluído tem dificuldade para escalar e para sofrer manutenções (BERG *et al.*, 2018). É comum o código estar no formato de monolito e a manutenção ser dificultada. O que era fácil, quando a empresa e a equipe eram pequenas, se torna insustentável com o crescimento. Neste momento, os requisitos não funcionais se tornam críticos e passam a ameaçar a existência da própria empresa.

Nem todos os requisitos de qualidade serão viáveis para implementação em um produto de *software* (WIEGERS; BEATTY, 2013). Depois de elicitados, certamente haverá uma análise de prioridade, e aqueles que forem identificados como mais prioritários irão direcionar as escolhas da equipe, seja em termos de alocação de pessoas, seja em termos de escolhas tecnológicas e arquiteturais.



Fique atento

Nem todos os produtos de *software* são sistemas de informação; vários deles, especialmente hoje, são sistemas ditos **embarcados**, ou seja, aqueles em que o *software* está dentro de produtos de *hardware*, como sistemas mais complexos. Sistemas dessa natureza envolvem dispositivos mecânicos, elétricos e eletrônicos, como sensores, atuadores, controladores, motores, baterias etc. Nesses casos, requisitos relacionados ao desempenho podem se tornar críticos.

Nos sistemas baseados em IoT (*Internet of Things*, ou Internet das Coisas), por exemplo, um ponto crucial é o consumo de energia. Algoritmos para esses casos devem ser otimizados e devem consumir pouca energia para processar.

Como identificar os requisitos não funcionais

Assim como os requisitos funcionais, os requisitos não funcionais podem ser obtidos pelo contato direto com os *stakeholders* em reuniões, entrevistas, *brainstormings*, ou por meio de questionários. Podem ainda ser descobertos quando se conduz uma observação no ambiente de execução. Nesses momentos, é possível, para o observador, identificar requisitos não funcionais relacionados ao uso do sistema.

Muitas vezes, os requisitos precisam ser obtidos a partir da análise de documentos, leis, regulamentações, guias etc. Nesses casos, os requisitos não funcionais podem aparecer de forma explícita ou implícita na documentação. Com a Lei Geral de Proteção de Dados Pessoais (LGPD), por exemplo, muito requisitos não funcionais foram explicitados. A consequência é que funcionalidades adicionais precisam ser implementadas em todos os sistemas do mundo. Uma delas, a mais óbvia, é alertar ao usuário, logo na instalação do produto, quais são as políticas de segurança e compartilhamento de dados que são aplicadas pelo fornecedor, exigindo que ele concorde com elas.



Saiba mais

Para saber mais sobre a Lei nº 13.709, de 14 de agosto de 2018, mais conhecida como Lei Geral de Proteção de Dados Pessoais (LGPD), consulte o *site* da Casa Civil do Governo Federal ou o Diário Oficial da União (DOU). Para obter informações sobre a lei internacional equivalente, pesquise por General Data Protection Regulation (GDPR).

Não existe uma fórmula mágica para fazer com que os requisitos não funcionais surjam, mas as atividades sugeridas por Wiegers e Beatty (2013, p. 265), apresentadas na Figura 1, podem ajudar. Elas foram baseadas em um documento disponibilizado *on-line* pela empresa Clarrus (2010).

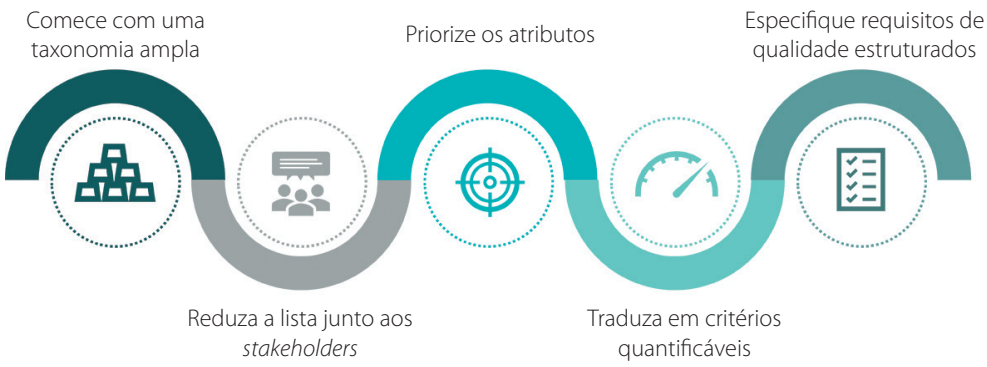


Figura 1. Elicitação de requisitos não funcionais.

Fonte: Adaptada de Wiegers e Beatty (2013).

Como se pode observar na Figura 1, os autores sugerem que a descoberta dos requisitos não funcionais comece partindo de uma **lista abrangente de atributos de qualidade**. Naturalmente, nem todos os tipos de *software* necessitarão de todos os atributos. Além disso, nem sempre se tem todos os recursos disponíveis para implementá-los, mas é melhor começar com uma lista maior do que correr o risco de deixar algum atributo importante de lado.

Como sugestão para a elaboração dessa lista, você pode utilizar as características e subcaracterísticas de qualidade que estão presentes na família de normas internacionais ISO/IEC 25000, mais especificamente o modelo de qualidade em uso e o modelo de qualidade de produto, que podem ser encontrados na norma internacional ISO/IEC 25010 (ISO/IEC, 2011).

É possível também usar a relação compilada por Wiegers e Beatty (2013), apresentada no Quadro 1.

Em seguida, deve-se engajar todos os *stakeholders* relevantes do produto para que possam **selecionar os atributos de qualidade** que são importantes para o projeto. Em um primeiro momento, pode ser que eles selecionem todos, ou quase todos os atributos da lista. Isso pode gerar uma lista inviável de ser implementada, seja pelas restrições de contexto, seja por conflitos entre os próprios atributos. Cabe ao analista de requisitos conduzir os *stakeholders* durante essas decisões.

Quadro 1. Alguns atributos de qualidade

Qualidade externa	Breve descrição
Disponibilidade	Extensão na qual os serviços do sistema estão disponíveis quando e onde são necessários
Instalabilidade	Quão fácil é instalar, desinstalar e reinstalar corretamente a aplicação
Integridade	A extensão na qual o sistema protege contra imprecisão e perda de dados
Interopera-bilidade	Quão fácil o sistema pode interconectar e trocar dados com outros sistemas e componentes
Desempenho	Quão rápido e previsivelmente o sistema responde às entra-das do usuário e outros eventos
Confiabilidade	Por quanto tempo o sistema roda antes de apresentar uma falha
Robustez	Quão bem o sistema responde a uma condição inesperada de operação
Proteção	Quão bem o sistema protege contra ferimentos e danos
Segurança	Quão bem o sistema protege contra acesso não autorizado à aplicação e seus dados
Usabilidade	Quão fácil é para as pessoas aprenderem, lembrarem e utilizarem o sistema
Qualidade interna	**Breve descrição**
Eficiência	Quão eficientemente o sistema usa os recursos do computador
Modificabilidade	Quão fácil é manter, modificar, melhorar e reestruturar o sistema
Portabilidade	Quão facilmente o sistema pode ser colocado em operação em outros ambientes operacionais
Reusabilidade	Em que extensão os componentes podem ser usados em outros sistemas
Escalabilidade	Quão facilmente o sistema pode crescer para tratar mais usuários, transações, servidores e outras extensões
Verificabilidade	Quão prontamente desenvolvedores e testadores podem confirmar que o *software* foi implementado corretamente

Fonte: Adaptado de Wiegers e Beatty (2013).

O próximo passo é realizar a priorização desses atributos. Uma forma de realizar essa atividade é usando uma tabela que combina os diversos atributos e faz análise sobre eles, dois a dois. Isso é importante para ajudar o analista de requisitos a focar as atividades de elicitação sobre os atributos relevantes, não gastando grande esforço em atributos que possam ser descartados. O Quadro 2 apresenta um exemplo adaptado de Wiegers e Beatty (2013) que reflete o que foi proposto por Clarrus (2010).

Quadro 2. Exemplo de quadro para priorização de atributos de qualidade

Qualidade externa	***Score***	**Disponibilidade**	**Integridade**	**Desempenho**	**Confiabilidade**	**Robustez**	**Segurança**	**Usabilidade**	**Verificabilidade**
Disponibilidade	1		^	<	^	^	^	^	^
Integridade	7			<	<	<	<	<	<
Desempenho	1				^	^	^	<	^
Confiabilidade	3					^	^	<	^
Robustez	4						^	^	<
Segurança	5							^	<
Usabilidade	3								^
Verificabilidade	4								

Fonte: Adaptado de Wiegers e Beatty (2013).

A tabela funciona de forma muito simples. Para cada par de atributos você faz a seguinte pergunta: se for para escolher um dos dois, qual é o mais importante? Ao entrar com um sinal de menor (<) indica que o atributo da linha é mais importante. Ao entrar com o sinal circunflexo (^) indica que o atributo da coluna é mais importante.

O exemplo apresentado no Quadro 2 é fictício. A leitura seria a seguinte: entre o atributo "disponibilidade" e "integridade", o símbolo aponta para cima, ou seja, para a "integridade". Isso significa que o atributo "integridade", para este contexto, é mais importante do que "disponibilidade". A lista de atributos é um recorte da lista abrangente apresentada no passo um e que foi refinada no passo dois. Esta matriz é útil para a tomada de decisão em momentos em que seja necessário priorizar um requisito em detrimento de outro.

A coluna do *score* é obtida pela soma de vezes em que o atributo é selecionado como prioridade. No caso do exemplo fictício do Quadro 2, nota-se que o atributo "integridade" foi mais importante do que todos os outros sete atributos, recebendo o *score* 7. Em seguida, vem o atributo "segurança", com 5 pontos, e assim por diante.

O objetivo do uso dessa tabela é obter uma visão consolidada sobre a prioridade dos requisitos não funcionais. É possível que elas sejam conflitantes entre os diversos *stakeholders*, e, nesse momento, cabe ao analista de requisitos resolver esses conflitos junto aos *stakeholders*.

A próxima etapa é traduzir os atributos em **critérios quantificáveis**. Geralmente os *stakeholders* usam palavras ambíguas para expressar os requisitos não funcionais, como amigável, rápido, confiável, robusto e assim por diante. Entretanto essas expressões geram interpretações imprecisas e, geralmente, são impossíveis de validar. Para chegar a critérios mais objetivos, será necessário que o analista de requisitos faça perguntas que os *stakeholders* saibam responder e na linguagem que eles entendem. Em vez de perguntar quão robusto o sistema deve ser, o analista deveria perguntar quantos usuários simultâneos estão previstos, por exemplo.

A última etapa é **estruturar os requisitos de qualidade**. Wiegers e Beatty (2013) recomendam que seja utilizado sempre o mnemônico SMART (*specific*, *measurable*, *attainable*, *relevant and time-sensitive*), ou seja, o requisito deve ser específico, mensurável, atingível, relevante e sensível ao tempo.



Fique atento

Se o testador não puder testar um requisito, então o requisito não está bom o suficiente.

Identificação de requisitos não funcionais em ambientes ágeis

Em ambientes de desenvolvimento ágil de *software*, a identificação dos requisitos não funcionais geralmente ocorre por meio de um *workshop* para discussão das histórias de usuário. As histórias de usuário vão focar nos requisitos funcionais, mas o seu complemento, que são os critérios de aceitação, irão incluir critérios referentes aos requisitos funcionais e também aos requisitos não funcionais. E é aí que eles, os requisitos não funcionais, entram em cena.

Patton (2014) apresenta algumas dicas para a condução de *workshops* para a discussão das histórias do usuário e dos critérios de aceitação, acompanhe:

- O *workshop* é, basicamente, uma conversa apoiada por diversas figuras e dados, ao final do qual a equipe deve chegar à conclusão de como será validado o que foi decidido construir.
- Antes do *workshop* é importante comunicar à equipe quais histórias do usuário serão discutidas.
- O ideal é um *workshop* pequeno, entre três a cinco pessoas. Patton (2014) recomenda que seja "do tamanho de conversa de jantar".
- Cerifique-se de ter incluído as pessoas certas: (1) alguém que entenda o usuário, geralmente um analista de negócios, o *product owner* ou alguém com conhecimento em experiência do usuário (UX); (2) desenvolvedores que entendam de código e que saibam analisar a viabilidade técnica do que está sendo proposto; (3) um analista de teste que fará as perguntas difíceis, quando a maioria será otimista. Outras pessoas podem ser incluídas, mas lembre-se de que muitas pessoas juntas podem tornar o *workshop* improdutivo.
- Mergulhe fundo para compreender: exatamente quem é o usuário; exatamente como ele vai usar o *software*; exatamente como ele deve se parecer (interface); exatamente como o *software* deve se comportar por trás da interface (regras de negócio e validação de dados); basicamente como o *software* será construído — para permitir estimativas.
- Entre em acordo sobre o que será construído, respondendo a duas perguntas: o que vamos checar para verificar o que foi construído; como vamos demonstrar o *software* quando formos revisar juntos.
- Converse e documente tudo o que ficou decidido, pode ser com desenhos, anotações e fotografias.
- Fale por meio de exemplos para materializar a história. Seja específico sobre situações e dados.
- Divida ou esvazie histórias, se necessário.

Utilizar *workshops* para definir os requisitos não funcionais, ou seja, os critérios de aceitação, pode ser uma excelente escolha. No entanto, dependendo da característica das pessoas envolvidas, pode não dar muito certo. Saiba como reconhecer quando o *workshop* não está funcionando verificando estes pontos (PATTON, 2014):

- Ninguém participa: alguém descreve o que é requerido e os outros escutam.
- Quando o foco fica apenas nos critérios de aceitação e não se discute a história e quem faz o que, e por quê.
- Quando se falha em identificar opções, sejam funcionais, sejam técnicas.



Fique atento

Identificar os requisitos não funcionais de *software* não exige uma técnica especial, mas sim uma atitude especial. É preciso que o analista de requisitos, ou papel equivalente, esteja atento para explorar, junto aos diversos *stakeholders*, quais são os pontos críticos do sistema e para onde a equipe de desenvolvimento deve envidar os seus esforços. Um *checklist* contendo os principais requisitos de qualidade pode ajudar na identificação desse tipo de requisito. A família de normas internacionais ISO/IEC 25000 pode ajudar.

2 Especificação de requisitos não funcionais de *software*

Quando falamos de requisitos funcionais logo nos vem à mente a especificação utilizando os diagramas de caso de uso e as especificações de caso de uso. Embora essas sejam duas ferramentas muito poderosas para a especificação de requisitos, elas não se aplicam aos requisitos não funcionais. Elas são boas para explicitar as funcionalidades.

Para as especificações de requisitos não funcionais, usamos alguma forma de documentação suplementar, que visa a explicitar os detalhes desses requisitos, de modo que possam seguir para as demais fases do desenvolvimento.

O Quadro 3 apresenta alguns exemplos de especificação de requisitos não funcionais, extraídos de Wiegers e Beatty (2013). Note que todos eles têm em

comum a característica de buscar trazer mais precisão para a declaração do requisito.

Quadro 3. Exemplos de especificação de requisitos não funcionais

Tipo	Identificador	Descrição
Disponibilidade	AVL-1	O sistema deve estar pelo menos 95% disponível nos dias de semana das 6:00 h às 24:00 h e pelo menos 99% disponível nos dias de semana entre 15:00 h e 17:00 h.
Instalabilidade	INS-1	Um usuário sem treinamento deve ser capaz de executar com sucesso a instalação inicial da aplicação em 10 minutos.
Integridade	INT-1	Depois de executar o *backup* de arquivos, o sistema deve verificar a cópia do *backup* em relação ao original e relatar qualquer discrepância.
Desempenho	PER-1	A autorização de um saque no caixa eletrônico não deve demorar mais do que 2 segundos.
Segurança	SEC-1	O sistema deve bloquear a conta do usuário depois da quarta tentativa de *login* sem sucesso consecutiva em um período de 5 minutos.
Usabilidade	USE-1	Todas as funções do menu Arquivo devem ser teclas de atalho definidas que usam a tecla Ctrl pressionada simultaneamente com outra tecla. Comandos do menu que também aparecerem no MS-Word devem usar as mesmas teclas de atalho *default* que o MS-Word usa.

Fonte: Adaptado de Wiegers e Beatty (2013).

Requisitos não funcionais nos ambientes ágeis

Quando uma empresa utiliza métodos ágeis, os requisitos são especificados no formato de histórias de usuário. De acordo com Patton (2014, p.91, tradução nossa), "Histórias foram assim nomeadas devido a como é esperado que elas

sejam usadas, não devido à forma como você está tentando escrevê-las". Uma história do usuário nada mais é do que um cenário de uso de um produto por um tipo específico de usuário.

Segundo Patton (2014), Roy Jeffrey, um dos criadores do XP, estabeleceu os 3 Cs da história de usuário: **C**artão, **C**onversa e **C**onfirmação, representados da Figura 2. O **cartão** é onde a história do usuário é escrita. A **conversa** refere-se ao fato de que a história nasce da intensa comunicação entre a equipe de desenvolvimento e o usuário. E, por fim, a **confirmação** representa os critérios de aceitação da história, que é onde os requisitos não funcionais são especificados. Também pode ser que você encontre lugares em que os requisitos não funcionais sejam registrados no próprio cartão, como se fosse uma história independente.



Figura 2. Os 3 Cs da história de usuário.

Fonte: Adaptada de Patton (2014).

3 Indicadores para avaliar requisitos não funcionais de *software*

Uma das formas de analisar se chegamos a um resultado é comparar o valor obtido com o valor planejado. Isso vale para qualquer aspecto da vida pessoal e profissional. Medidas fazem parte de nossa vida desde que nascemos: nasceu

dentro da faixa de peso ideal? Cresceu o suficiente no primeiro mês? Está no tamanho ideal para a idade?

Se você quiser saber se foi bem em uma prova, você compara o resultado ideal (o 10,0) com o resultado que você obteve — isso vai lhe mostrar o quanto você está distante do ideal. Se você quiser comparar o seu desempenho com o dos colegas, vai analisar o que você tirou em relação à média dos seus colegas. Se você estiver em um treinamento para fazer uma maratona, vai analisar quantos quilômetros consegue correr em determinado tempo. Se quiser emagrecer, vai estabelecer um peso ideal e um caminho para chegar lá.

Medições também fazem parte da nossa vida profissional: quanto tempo precisamos para entregar esta funcionalidade? Quantas pessoas são necessárias para este projeto? Quanto tempo falta para terminar o projeto? Qual é a produtividade média da equipe? Qual é a gravidade deste risco? Quanto tempo levaríamos para implementar todo o *backlog* do produto?

Só é possível controlar aquilo que se consegue medir. Só é possível avaliar se houver algum padrão para comparar. Só é possível planejar com certa precisão se houver medidas confiáveis nas quais se basear.

E como medir a qualidade de um produto de *software*? Como garantir se o que foi entregue é o que o usuário queria? Se estivermos falando de funcionalidades, podemos contar quantas funcionalidades foram entregues em relação a quantas funcionalidades foram solicitadas. Mas será que isso é suficiente? E se a funcionalidade não apoiar o usuário adequadamente em suas tarefas, conta como funcionalidade entregue com qualidade?

A grande questão é: como garantir que aquilo que vamos entregar atenderá às expectativas dos usuários do produto? Inicialmente, investindo na elicitação de requisitos, com especial atenção para os requisitos não funcionais. Em seguida, identificando indicadores que podem ser perseguidos pela equipe técnica para garantir que as necessidades dos usuários serão atendidas na medida certa.

Conceitos de medição

Existem termos que têm um significado específico quando se trata de medição. Vamos adotar aqui os conceitos utilizados na norma internacional ISO/IEC 15939 (ISO/IEC, 2007), que é a norma da ISO/IEC que descreve o processo de medição. Os termos relevantes se encontram no Quadro 4.

Quadro 4. Definição de termos para medição

Termo	Significado
Atributo	Propriedade ou característica de uma entidade que possa ser distinguida quantitativa ou qualitativamente por seres humanos ou meios automatizados
Dados	Coleta de valores atribuídos a medidas base, medidas derivadas e/ou indicadores
Entidade	Objeto que deve ser caracterizado medindo seus atributos
Escala	Conjunto ordenado de valores, contínuo ou discreto, ou um conjunto de categorias para as quais o atributo está mapeado
Função de medição	Algoritmo ou cálculo realizado para combinar duas ou mais medidas básicas
Indicador	Medida que fornece uma estimativa ou avaliação de atributos especificados derivados de um modelo com relação a necessidades de informação definidas
Medição	Conjunto de operações com o objetivo de determinar uma medida
Medida	Variável à qual um valor é atribuído resultante de uma medição
Medida básica	Medida definida em termos de um atributo e método para quantificá-lo
Medida derivada	Medida que é definida como uma função de dois ou mais valores de medidas básicas
Medir	Fazer uma medição
Procedimento de medição	Conjunto de operações, descrito especificamente, utilizado no desempenho de determinada medição de acordo com determinado método
Valor	Resultado numérico ou categórico atribuído a uma medida básica, medida derivada ou indicador

Fonte: Adaptado de ISO/IEC (2007).

A Figura 3, adaptada da norma ISO/IEC 15939 (ISO/IEC, 2007), apresenta os relacionamentos entre os elementos envolvidos na medição. Para que eles possam ser mais bem compreendidos, vamos utilizar um exemplo prático e fácil. Vamos supor que a **necessidade de informação** de um *stakeholder* seja identificar se a produtividade de sua equipe de desenvolvimento está compatível com a produtividade média do mercado.

Para atender a essa necessidade de informação vamos começar identificando as entidades envolvidas e quais são os atributos (características) dessas entidades que nós precisamos medir para atender à necessidade de informação. Nesse caso, a **entidade** pode ser a tarefa realizada e os **atributos** podem ser o tamanho da tarefa e o tempo necessário para desenvolver a tarefa.

Em seguida definimos qual é o **método de medição** envolvido. Neste caso precisamos saber como obter o valor do atributo de tamanho da tarefa e o valor do atributo do tempo. Se a tarefa for uma história de usuário, podemos utilizar o conceito de *story points* (pontos de história do usuário). Para o atributo tempo, podemos assumir que seja o tempo decorrido desde o início até o término da história do usuário. Portanto, nossas **medidas básicas** serão quantidade de pontos de história de usuário e tempo de desenvolvimento da história.

Muitas vezes temos que nos valer de métodos de medição mais subjetivos (que envolvem o julgamento humano). Devemos evitá-los e dar sempre preferência a métodos mais objetivos, ou seja, que envolvem uma medição (contagem) a ser realizada de forma manual ou automatizada.

A **função de medição** será a forma como vamos combinar as duas medidas básicas para gerar a **medida derivada**. Nesse caso vamos considerar que a medida derivada será a produtividade e que a função de medição será a razão entre a quantidade de pontos de história de usuário pelo tempo de desenvolvimento da história.

Em seguida estabelecemos o **modelo de análise**, que é a forma como vamos analisar a medida derivada que acabamos de obter. Neste nosso exemplo, podemos entender que seria a forma como vamos interpretar a produtividade obtida. Aqui estão envolvidos os critérios de decisão, que poderiam envolver o seguinte: qual é o valor médio da produtividade no mercado para este tipo de produto, com esta tecnologia? Essa seria a base para poder interpretar a produtividade que foi obtida usando nossa função de medição.

O que se obtém é um **indicador**, que pode nos dizer se a produtividade está abaixo do mercado, se a produtividade está na média do mercado ou se a produtividade está acima do mercado. Isso nos permite realizar a **inter-**

pretação da informação, que é a explicação relacionada com a informação quantitativa expressa pelo indicador, em uma linguagem que a pessoa que vai usar a medição compreenda.

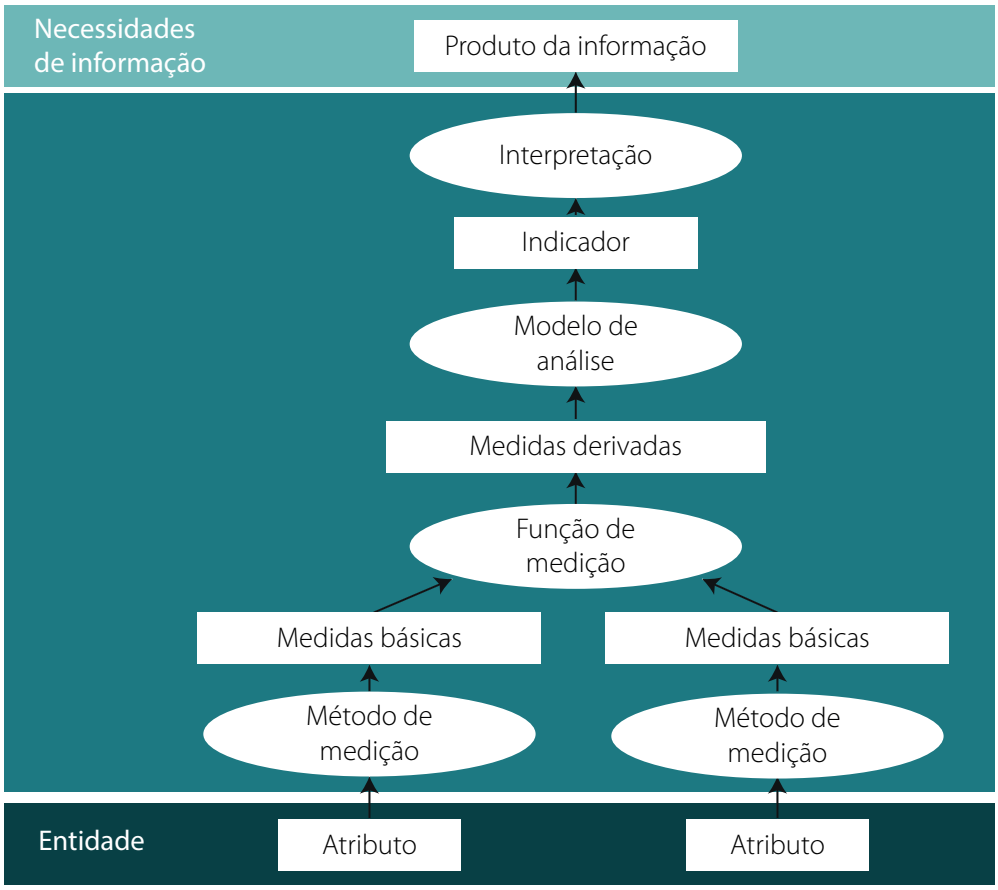


Figura 3. Modelo de Informação de Medição da ISO/IEC 15939.

Fonte: Adaptada de ISO/IEC (2007).



Saiba mais

A norma internacional ISO/IEC 15939 ISO/IEC (2007) apresenta mais detalhes sobre o processo de medição. Exemplos são fornecidos no Anexo A da norma, como forma de esclarecer os conceitos em contextos diferentes.

Como definir indicadores de qualidade de produto

Na seção anterior vimos os conceitos de medição, vimos como esses conceitos se relacionam e vimos também um exemplo aplicado a uma medição do cotidiano das empresas — a produtividade. Agora vamos ver como aplicar isso à qualidade do produto de *software*, ou seja, aos atributos que estão envolvidos em determinar se os requisitos não funcionais foram atendidos.

A família de normas internacionais da ISO/IEC que trata da qualidade dos produtos de *software* é a família 25000 (ISO/IEC, 2014). Entre esse conjunto abrangente de normas, podemos destacar as seguintes:

- ISO/IEC 25022: Medição da qualidade em uso.
- ISO/IEC 25023: Medição da qualidade de produto de sistemas e *software*.
- ISO/IEC 25024: Medição da qualidade de dados.

A ISO/IEC 25022 (ISO/IEC, 2016a) trata da medição de aspectos relacionados à qualidade em uso, ou seja, aqueles aspectos que são percebidos pelo usuário, quando ele usa o produto no ambiente-alvo. Seria, por exemplo, quando você utiliza um aplicativo no seu celular para pedir comida. Neste caso, haverá dois aspectos envolvidos na sua percepção de qualidade: um está relacionado ao serviço prestado e o outro, ao produto de *software* em si. E é bem comum confundir os dois!



Exemplo

Vamos imaginar que você queira pedir uma pizza individual e o aplicativo só mostre pizzas grandes. Esse certamente será um problema que vai deixar você muito insatisfeito, mas esse não é um problema do *app* que você está usando, mas sim do serviço que está sendo prestado via *app*. O problema é da pizzaria, que não oferece a pizza do tamanho que você precisa.

Agora, se na hora que você for fechar o pedido, tiver dificuldade para conseguir localizar onde pode trocar o endereço de entrega, mesmo essa funcionalidade existindo no *app*, então esse é um problema de usabilidade do aplicativo. Ele provavelmente não tem uma interface intuitiva, uma vez que você não consegue localizar uma funcionalidade. Esse sim é um requisito não funcional relacionado à usabilidade do produto e não ao serviço.

Mas há limites menos óbvios para o usuário entre o que é um problema do produto e o que é um problema do serviço. Vamos continuar no exemplo da pizza: imagine que só é oferecida a forma de pagamento com cartão de crédito. Este é um problema do serviço ou do produto de *software*? Na verdade, depende. Pode ser que o aplicativo ofereça diversas formas de pagamento e o estabelecimento comercial optou por configurar apenas a opção de pagamento em cartão de crédito. Neste caso, é uma limitação do negócio e não do aplicativo. Já se o aplicativo não permitir o cadastramento de outra forma de pagamento, por mais que o estabelecimento permita, então é uma limitação do aplicativo, que causa desconforto ao usuário.

Certamente você já deve ter experimentado diversos aplicativos no seu celular. Alguns você achou fáceis e agradáveis de utilizar e outros você abandonou o uso porque não gostou deles. Às vezes os motivos de não ter gostado são fáceis de identificar, por exemplo: achei a tela muito poluída e ruim de navegar, os controles não estavam fáceis de localizar, o aplicativo vivia travando, não me sentia seguro em informar meus dados bancários etc. Mas, às vezes, você não consegue saber com certeza porque não gosta deste ou daquele aplicativo, e é aí que está o problema: **utilizar um *software* é uma experiência que envolve diversas percepções**, muitas delas difíceis de precisar, pois são mais sutis e subjetivas.

O que tentamos fazer ao definir medidas para avaliar os requisitos não funcionais é retirar o máximo que conseguimos das subjetividades e tornar a avaliação mais precisa. A ISO/IEC 25022 (ISO/IEC, 2016a) nos ajuda apresentando as medidas que podem ser utilizadas para avaliar, de forma objetiva, a qualidade em uso de produtos de *software*.

A Figura 4 apresenta as características relacionadas à qualidade em uso, para as quais a norma apresenta as propostas de medidas.



Figura 4. Características de qualidade em uso tratadas na ISO/IEC 25022.
Fonte: Adaptada de ISO/IEC (2016a).

A norma propõe um grupo de medidas referentes à **efetividade**, que são aquelas que avaliam a precisão e a completude com que o produto apoia as atividades do usuário. A norma sugere que podem ser usadas medidas como as seguintes:

- percentual de tarefas que foram completadas sem ajuda, em relação ao total de tarefas que se tentou realizar;
- percentual de objetivos que foram atingidos sem ajuda;
- quantidade de erros cometidos pelo usuário ao realizar uma tarefa;
- percentual de tarefas que tiveram erros cometidos pelo usuário;
- proporção de usuários cometendo erros em tarefas.

É fácil imaginar que se o usuário comete erros realizando a tarefa no *software*, significa que o *software* não está ajudando o usuário a evitar esses erros. Se utilizarmos as medidas definidas acima, podemos identificar o tamanho do impacto que isso pode acarretar à experiência do usuário.

E como evitamos erros cometidos pelo usuário? Por exemplo, quando colocamos uma mensagem do tipo "Confirma a operação?", estamos dando ao usuário a oportunidade de conferir o que ele realizou e de voltar atrás. É uma forma de chamar a atenção do usuário em pontos críticos, onde um erro

pode ter consequências mais graves. Esse é um recurso muito usado quando estamos finalizando uma operação bancária, por exemplo.

Medidas de **eficiência** também são propostas pela norma. Essas são as medidas que avaliam o uso dos recursos para que o usuário atinja os seus objetivos: podemos usar medidas simples, como o tempo que o usuário leva para fazer uma tarefa, ou podemos usar medidas mais complexas, como as listadas a seguir:

- Eficiência em relação ao tempo — quantidade de tarefas que ele faz em determinado tempo (por exemplo, quantos novos cadastros ele consegue fazer por hora).
- Eficiência em relação ao custo — custo para realizar as tarefas.
- Proporção de tempo que o usuário está executando uma tarefa útil — tempo que o usuário realiza a tarefa real, em comparação ao tempo que ele gasta esperando o sistema se recuperar de um erro, por exemplo, ou o tempo que ele gasta pedindo ajuda.
- Ações desnecessárias.
- Consequências da fadiga — como a produtividade cai com o uso prolongado.

Uma medida muito interessante que pode ajudar em dois aspectos diferentes é a quantidade de ações desnecessárias que um usuário realiza para completar uma tarefa. Por exemplo, a quantidade de cliques errados que ele deu antes de conseguir chegar no item correto que ele precisava. Isso pode apontar uma ineficiência (tempo gasto em tarefas não úteis) e também um problema na usabilidade — se ele tem que clicar em diversos itens incorretos antes de achar o item correto, a interface não está intuitiva.

Um terceiro aspecto da qualidade em uso para o qual a norma define medidas é a **satisfação**. Ela visa a medir o grau de satisfação do usuário quando utiliza o *software*. Na norma, as medidas de satisfação estão divididas em subcaracterísticas, que são as seguintes: utilidade, confiança, prazer (experiência do usuário), conforto (ergonomia).

Utilidade pode ser medida da seguinte forma:

- medida de satisfação geral;
- medida de satisfação com uma funcionalidade específica;
- percentual de usuários que usa uma função em relação ao total que poderia utilizá-la;

- percentual de usuários reclamando do *software*;
- percentual de reclamações dos usuários em relação a uma funcionalidade específica em relação às reclamações totais.

As subcaracterísticas de **confiança**, **prazer (experiência do usuário)** e **conforto (ergonomia)** podem ser medidas por meio de uma pesquisa de satisfação junto aos usuários do produto.

A quarta característica que a norma nos ajuda a gerar medidas em relação à qualidade em uso é o fato de o *software* **ser livre de riscos** (do original, em inglês *freedom from risk*). Riscos podem estar relacionados às subcaracterísticas de riscos econômicos, riscos de saúde e segurança e riscos ambientais.

- Os **riscos econômicos** estão mais relacionados a aspectos de negócio, como o retorno sobre o investimento (ROI), o momento para iniciar o ROI, a lucratividade com o uso do produto e o nível de serviço dos usuários, quantos visitantes de um *website* se tornam clientes, vendas para um cliente e prejuízos decorrentes do produto.
- Os **riscos de saúde e segurança** se referem, como o próprio nome sugere, aos relatos de problemas de saúde ou de segurança decorrentes do uso do *software*. Esses indicadores são especialmente importantes quando estamos desenvolvendo sistemas de missão crítica que podem ocasionar perdas de vidas ou danos à sua segurança e saúde.
- Finalmente, os **riscos ambientais** se referem a possíveis danos ao ambiente que o produto de *software* possa ocasionar. Dependendo do tipo de sistema, isso pode estar relacionado à poluição, poluição auditiva ou aquecimento global, por exemplo.

A quinta e última característica que a norma aborda é referente à **cobertura do contexto.** Neste caso, são abordados dois aspectos: completude do contexto e flexibilidade. A medida de **completude do contexto** é mais específica e se refere ao quanto o contexto é coberto pelo *software.* Já as medidas sugeridas para a flexibilidade incluem o seguinte:

- contexto de uso flexível;
- flexibilidade do produto;
- independência de proficiência.



Fique atento

Apresentamos nesta seção um resumo de algumas das sugestões realizadas pela norma internacional ISO/IEC 25022 (ISO/IEC, 2016a). Para maiores detalhes, a norma deverá ser consultada.



Referências

BERG, V. *et al*. Software startup engineering: a systematic mapping study. *Journal of Systems and Software*, v. 144, p. 255–274, out. 2018. Disponível em: https://www.researchgate.net/profile/Ilias_Pappas/publication/325858070_Software_Startup_Engineering_A_Systematic_Mapping_Study/links/5c3dbfb3458515a4c726f927/Software-Startup-Engineering-A-Systematic-Mapping-Study.pdf. Acesso em: 24 abr. 2020.

CLARRUS. *Software quality attribute*: following all the steps. [*S. l.*], 2010. Disponível em: https://www.clarrus.com/documents/Software%20Quality%20Attributes.pdf. Acesso em: 24 abr. 2020.

ISO/IEC. *ISO/IEC 25000:2014*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), guide to SQuaRE. Switezland: ISO, 2014.

ISO/IEC. *ISO/IEC 25022:2016*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), measurement of quality in use. Switezland: ISO, 2016a.

ISO/IEC. *ISO/IEC 25023:2016*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), measurement of system and software product quality. Switezland: ISO, 2016b.

ISO/IEC. *ISO/IEC 25024:2015*: systems and software engineering, systems and software quality requirements and evaluation (SQuaRE), measurement of data quality. Switezland: ISO, 2015.

ISO/IEC. *ISO/IEC/IEEE 15939:2017*: systems and software engineering, measurement process. Switzerland: ISO, 2007.

PATTON, J. *User story mapping*: discover the whole story, build the right product. Sebastopol: O'Reilly, 2014.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3rd ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

Parte 3

Verificação de Requisitos de Software

O conteúdo deste livro
é disponibilizado
por SAGAH.

SAGAH
SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Verificação de requisitos de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Reconhecer a importância da verificação de requisitos de *software*.
- Aplicar *checklists* de verificação de requisitos funcionais e não funcionais de *software*.
- Criar casos de teste para a verificação de requisitos funcionais e não funcionais de *software*.

Introdução

Todos sabemos que o mercado tem se tornado cada vez mais exigente com relação à qualidade dos produtos de *software*. Isso se dá em parte porque existe uma grande quantidade de *softwares* disponíveis para resolver diferentes tipos de problemas do cotidiano. As lojas virtuais oferecem uma infinidade de aplicativos para celular, tanto para iOS quanto para Android, que estão disponíveis a apenas um toque de distância. Assim como é fácil baixar e começar a usar, também é fácil desinstalar e escolher outro aplicativo para substituí-lo.

Essa intensa exposição à tecnologia na vida pessoal torna os usuários mais exigentes inclusive com os *softwares* que ele utiliza no trabalho. Nesse ambiente, entretanto, não é tão simples substituir um produto por outro. Não se instala e desinstala um *software* corporativo, como, por exemplo, um sistema de gestão integrada, com a mesma facilidade com que se substitui um aplicativo para acompanhar exercícios físicos pelo *smartwatch*.

E como garantir que os produtos de *software* tenham a qualidade esperada e proporcionem uma experiência extraordinária ao usuário? A resposta não é trivial e envolve todas as etapas do ciclo de desenvolvimento de *software*, desde as primeiras conversas sobre o escopo do projeto até a entrega de uma versão ou do produto final ao usuário. Não importa que tipo de método estejamos utilizando para desenvolver, o objetivo é o mesmo: fazer uma entrega de qualidade.

Diferentemente da manufatura, que emprega muitas etapas automatizadas, o desenvolvimento de *software* é uma atividade essencialmente cognitiva, que apresenta desafios muito mais complexos, pois erros humanos podem ser cometidos durante todas as fases. E o que podemos fazer para reduzir os efeitos nocivos desses erros? Utilizar técnicas de verificação e validação ao longo das atividades de todo o ciclo de vida. E isto começa pela etapa de requisitos.

Neste capítulo você vai estudar a verificação de requisitos de *software* e sua importância para o ciclo de desenvolvimento. Vai ver como utilizar *checklists* de verificação de requisitos funcionais e não funcionais e também como criar casos de teste para a verificação de requisitos funcionais e não funcionais.

1 Importância da verificação de requisitos de *software*

A engenharia de requisitos é a especialidade da engenharia de *software* que trata de todos os temas relacionados aos requisitos. Podemos compreender, de acordo com o SWEBOK (Software Engineering Body of Knowledge), que a engenharia de requisitos é "uma forma sistemática de tratar os requisitos" (BOURQUE; FAIRLEY, 2014, documento *on-line*).

Etapas da engenharia de requisitos

Ainda de acordo com o SWEBOK (BOURQUE; FAIRLEY, 2014, documento *on-line*): "A área de conhecimento de Requisitos de Software (KA) preocupa-se com a obtenção, análise, especificação, e validação de requisitos de *software* bem como o gerenciamento de requisitos durante todo o ciclo de vida do produto de *software*".

Vamos relembrar essas etapas referentes à engenharia de requisitos por meio da Figura 1. Considere o semicírculo mais externo como as atividades da engenharia de requisitos e o semicírculo interno como o desenvolvimento em si, ou seja, a implementação. Permeando todo o processo está o gerenciamento de requisitos.

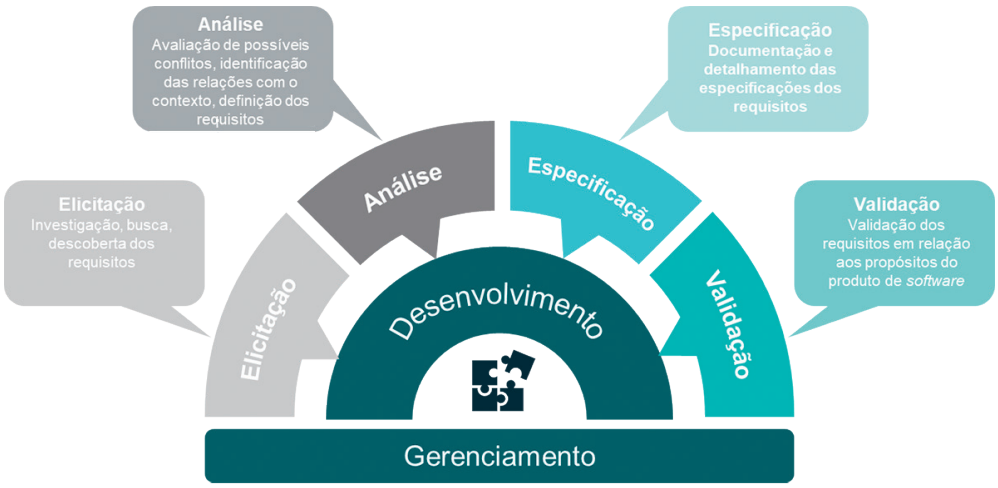


Figura 1. Etapas da engenharia de requisitos.

Conforme você pode observar na Figura 1, a primeira etapa se refere à **elicitação de requisitos**. A elicitação de requisitos é "[…] o primeiro estágio para construir uma compreensão sobre o problema que o *software* deve resolver" (BOURQUE; FAIRLEY, 2014, documento *on-line*). Nela são identificadas as fontes de informação e são selecionadas as técnicas de elicitação mais adequadas, de acordo com o contexto do projeto. Após a seleção, é possível aplicar essas técnicas para obter os requisitos, sejam eles funcionais ou não funcionais.

Embora estejamos utilizando a expressão "etapa de elicitação", é bom lembrar que isso não significa que é necessário que elicitar o conjunto completo de requisitos em um mesmo momento e nem que seja preciso chegar a níveis similares de detalhamento para cada requisito. Pode-se trabalhar de forma iterativa e incremental, como nas *sprints* dos métodos ágeis, nas quais os requisitos do *product backlog* vão sendo detalhados à medida que são necessários para alocação à *sprint*.

A segunda etapa se refere à **análise de requisitos**, que é o momento em que nos aprofundamos nos requisitos e buscamos identificar inconsistências e conflitos entre os requisitos. Identificamos também as fronteiras do sistema e com quem ele deve interagir. Segundo Vazquez e Simões (2016), o objetivo da elicitação é encontrar as peças do quebra-cabeças e o objetivo da análise é montá-lo.

A terceira etapa é a **especificação de requisitos**. Como o próprio nome sugere é o momento em que nos dedicamos a documentar os requisitos de modo que possam ser utilizados pelas etapas seguintes do desenvolvimento, evitando ambiguidades e retrabalho. O nível necessário de especificação varia de acordo com a criticidade do *software* para o ambiente onde ele será utilizado, da experiência da equipe desenvolvedora, dos riscos envolvidos etc. Níveis diferentes de especificação podem ser necessários para cada requisito do produto. Em sistemas nos quais o *software* é apenas um componente da solução, é comum que as especificações sejam mais robustas e complexas, envolvendo diversos tipos de documentos: definição do sistema, especificação de requisitos do sistema e especificação de requisitos do *software*.

Finalmente, a quarta etapa é chamada genericamente de **validação de requisitos**, embora, na verdade, ela englobe tanto a validação quanto a **verificação de requisitos**.



Fique atento

Verificação e validação são termos diferentes! De acordo com a norma internacional ISO/IEC/IEEE 12207 (ISO/IEC/IEEE, 2017, p. 10–11):

- **Validação** é "a confirmação, por meio do fornecimento de evidência objetiva, que o requisito foi atendido para um uso ou aplicação pretendidos específicos".
- **Verificação** é a "confirmação, por meio do fornecimento de evidência objetiva, que os requisitos especificados foram atendidos".

Ainda, de acordo com o CMMI-DEV 2.0 (CMMI INSTITUTE, 2018, p. 525), a diferença entre os termos é a seguinte:

- **Validação** "demonstra que a solução vai atender seu uso pretendido no ambiente alvo, isto é, 'você está construindo a coisa certa'".
- **Verificação** "endereça que o produto de trabalho ou a solução refletem adequadamente os requisitos especificados, isto é, 'você está construindo certo'".

De acordo com Bourque e Fairley (2014, p. 1-11),

> [...] os requisitos podem ser **validados** para assegurar que o engenheiro de *software* compreendeu os requisitos; é também importante **verificar** que o documento de requisitos está em conformidade com os padrões da empresa e que ele é compreensível, consistente e completo.

É comum dizermos que a verificação analisa se o produto foi construído certo, ou seja, de forma correta; e que a validação analisa se foi construído o produto certo, ou seja, aquele que os *stakeholders* desejavam (CMMI INSTITUTE, 2018).



Saiba mais

De acordo com o modelo CMMI-DEV 2.0 (CMMI INSTITUTE, 2018), a verificação e a validação na engenharia de *software* envolvem simulação, estudos de rastreabilidade, revisões ou auditorias funcionais, revisões ou auditorias físicas, revisões por pares, demonstrações, protótipos, revisões formais, testes de módulo, regressão e integração de sistema.

Finalmente, a próxima etapa se refere ao **gerenciamento de requisitos**. Nesta etapa vamos nos preocupar com como tratar os requisitos ao longo do seu ciclo de vida e como analisar e gerenciar as solicitações de mudanças que, inevitavelmente, ocorrerão.

Neste capítulo vamos nos concentrar na etapa de validação de requisitos, mais especificamente nas atividades de verificação de requisitos. Atividades de validação propriamente ditas não serão tratadas neste capítulo.

Importância da verificação de requisitos

À medida que os *softwares* foram se tornando mais complexos, mais integrados e mais críticos para todas as atividades cotidianas, mais importante ficou a garantia do seu correto funcionamento. Essa maior proximidade com a tecnologia vem tornando os usuários cada vez mais exigentes.

A grande questão é que há muitos pontos no ciclo de desenvolvimento nos quais erros que podem levar a falhas gigantescas de *software*. Isso ocorre porque as atividades envolvidas são de ordem cognitiva. A cada troca de mãos do requisito, ou seja, a cada troca de responsabilidade, um novo defeito pode estar sendo introduzido, como naquela brincadeira infantil do telefone sem fio.

Se no início do projeto o analista de requisitos não envolver todos os *stakeholders* relevantes para atuar como fonte de informação de requisitos, pode ser que um requisito muito importante não seja descoberto e permaneça invisível até as fases mais avançadas, levando ao retrabalho e a custos adicionais de desenvolvimento. Esta é a primeira passagem de bastão do requisito, ou seja, é a primeira transição. Ele sai do ambiente dos *stakeholders* e passa para o ambiente do projeto, via processo de elicitação de requisitos conduzido pelo analista de requisitos.

Caso os requisitos, uma vez elicitados, não estejam descritos de forma clara e não ambígua, eles podem ser implementados de forma incorreta. Esta é a segunda passagem de bastão do requisito. Ele sai das mãos do analista de requisitos e passa para as mãos da equipe técnica, que pode ser composta por arquitetos, projetistas e desenvolvedores, que irão projetar a arquitetura da aplicação, as interfaces e desenvolver o código.

Por fim, uma terceira passagem de bastão, ou seja, troca de responsabilidade, acontece quando a equipe de testes define os casos de teste que irão avaliar se o produto foi construído de acordo com o que estava especificado. Nesse caso o requisito sai das mãos do analista de requisitos e da equipe técnica e vai para as mãos dos testadores. Aqui podem acontecer problemas de interpretação que levam à construção de casos de teste ineficazes para o teste do produto.

Não podemos nos esquecer da última troca de bastão, que irá ocorrer quando o produto for implantado no ambiente-alvo e utilizado pelos seus usuários para cumprir as funções para as quais ele foi especificado.

Todas essas diversas trocas de responsabilidade podem levar a erros que se refletirão no ambiente de operação e podem trazer consequências diversas, como o retrabalho e seus custos associados ou falhas na operação, que podem gerar riscos de ferimentos ou de prejuízos financeiros. Ao longo da história da computação, diversos foram os casos de problemas não detectados em fases iniciais do desenvolvimento e que foram propagados até gerar grandes desastres. Vários deles você certamente já ouviu falar.

Exemplo

Diversos são os exemplos de falhas de grande impacto ocasionadas por problemas em *software*, muitos deles oriundos da etapa de requisitos. Vamos relembrar alguns exemplos famosos:

- O projeto de construção do **aeroporto de Denver**, nos Estados Unidos, subestimou a complexidade do algoritmo de controle das esteiras de bagagem, gerando dificuldade de integração entre o *hardware* e o *software*, e levando a prejuízos na ordem de U$ 1,1 milhão ao dia para a cidade de Denver (CALLEAM CONSULTING, 2008).
- Em 2019, a Boeing teve que recolher toda a sua frota de 387 aviões do modelo **Boeing 737 MAX**, que estava em 59 companhias áreas, devido a um problema no novo *software* (BOEING..., 2019).
- O foguete **Ariane 5** explodiu em pleno ar devido a uma diferença de tamanho de campos de dados entre ele e o seu antecessor, cujo *software* foi reaproveitado. O prejuízo foi de US$ 370 milhões (HINKEL, [201-?]).

Muitas dessas falhas poderiam ter sido evitadas se procedimentos adequados de qualidade tivessem sido adotados ao longo do ciclo de vida, conforme ilustra a Figura 2, por meio das atividades de Verificação 1 a 5, representadas pelos triângulos. Note que está representado apenas um ciclo de vida genérico e bastante simplificado, que pode ser uma *sprint*, por exemplo.

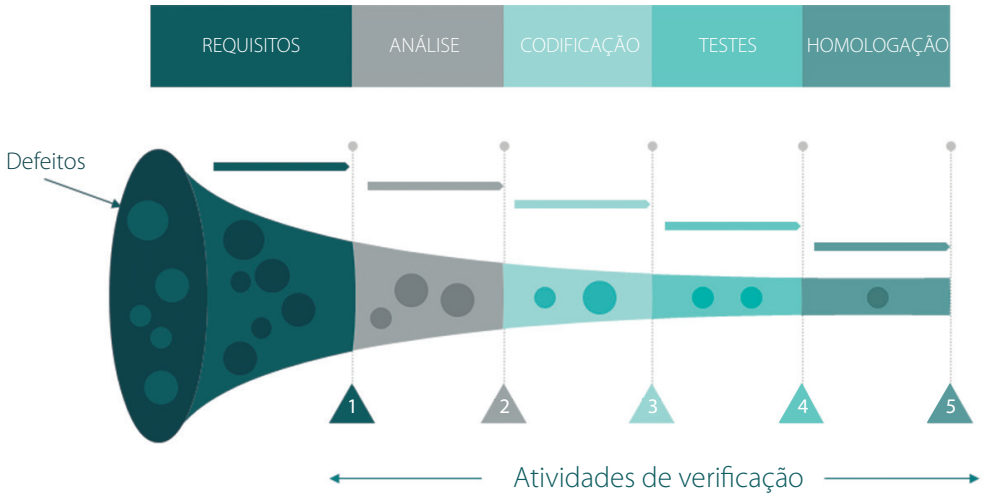


Figura 2. Defeitos inseridos e removidos ao longo do ciclo de desenvolvimento.

Como você pode observar, os defeitos estão presentes desde o início. Se não tivéssemos introduzido as atividades de verificação intermediárias, teríamos chegado à fase de testes com a quantidade inicial de defeitos (início do cone na figura). Possivelmente, esses defeitos ainda teriam sido amplificados, considerando que um defeito em um requisito pode implicar em defeitos em outros requisitos também.

Quando introduzimos a atividade de Verificação 1, os defeitos provenientes da etapa de requisitos são reduzidos. Note que não estamos aqui falando que todos os requisitos do produto deveriam estar definidos, ou seja, não estamos nos referindo a um ciclo cascata. Estamos nos referindo ao conjunto de requisitos que deveriam estar esclarecidos para uma versão ou *sprint*.

Ao encerrarmos a fase de análise, é inserida a atividade de Verificação 2 e, novamente, os defeitos são reduzidos, de modo a passar para a codificação um requisito que já esteja adequado para implementar. Se as atividades de Verificação 1 e Verificação 2 não tivessem sido realizadas, a codificação receberia todos os defeitos provenientes das fases anteriores.

Ao encerrarmos a codificação, esperamos que o desenvolvedor tenha realizado os testes de unidade ou testes unitários, que também são uma atividade de verificação. Ao final dessa fase, a atividade de Verificação 4 pode ser instituída tanto por meio de revisões de código quanto por meio de atividades de teste integrado.

Ao chegarmos na homologação, que é realizada pelo usuário, é esperado que esses defeitos tenham sido significantemente reduzidos. Note que, neste modelo fictício, se as atividades intermediárias não tivessem sido inseridas, a homologação receberia a quantidade inicial de defeitos, possivelmente ainda ampliada por outros inseridos como decorrência destes.

Diversas técnicas podem ser aplicadas para evitar que os erros se propaguem ao longo das fases, funcionando como uma espécie de filtro, como vimos na Figura 2. Quanto maior a quantidade de filtros e quanto mais finos eles forem, menos erros de propagarão. O problema é que esses filtros têm um custo, o chamado **custo da qualidade**, que deve ser cuidadosamente analisado para determinar o ponto de equilíbrio ideal.



Fique atento

Cada ponto de verificação precisa prover insumos para que o processo de desenvolvimento como um todo possa ser melhorado e a **causa raiz** dos erros possa ser identificada, de modo que no futuro os **defeitos possam ser prevenidos**.

Quando um defeito é detectado, ele precisa retornar para ser consertado, e isso é um retrabalho. Por isso é tão importante analisar os resultados provenientes das atividades de verificação. Portanto, *insights* sobre como melhorar o processo para evitar os erros constituem uma das saídas mais valiosas da análise de um processo de verificação.

A definição dos pontos onde essas intervenções são necessárias dependerá, primeiramente, da criticidade que uma falha representa para determinado sistema. Se a consequência for a perda de vidas, certamente os investimentos em qualidade serão todos compensados. Se a consequência for apenas uma pequena parada de um sistema que não tenha consequências críticas, então é possível que não se invista tanto nessas atividades de qualidade, priorizando esforços em outros pontos mais críticos.

Como realizar a verificação de requisitos

Os requisitos podem ser verificados de diversas formas. A primeira delas, naturalmente, é a leitura crítica dos artefatos, que é realizada pelo próprio autor, ou seja, o analista de requisitos, analisando pontos de ambiguidade, de inconsistência e de falta de completude nas definições. Para isso, ele pode ou não se apoiar em um *checklist,* que é um instrumento que funciona como uma espécie de lembrete, para que nenhum ponto seja esquecido.

Outras formas envolvem a revisão por pares, ou seja, a revisão que é realizada, como o próprio nome diz, pelos pares do analista de requisitos. Esses pares podem ser outros analistas de requisitos ou pessoas técnicas da equipe. São consideradas revisões por pares as listadas a seguir (CMMI PRODUCT TEAM, 2010):

- inspeções;
- *walkthroughs* estruturados;
- refatoração deliberada;
- programação em pares (*pair programming*).

Da mesma forma que a autoavaliação, a avaliação por pares pode ou não ser apoiada por um *checklist*. Na próxima seção veremos como elaborar e utilizar um *checklist*.

Outra forma, e a mais utilizada, são os testes de *software*. Testes são realizados para identificar erros que já estão materializados no código. Abordaremos esse assunto mais adiante neste capítulo.

2 *Checklists* de verificação de requisitos de *software*

Conforme apresentado anteriormente, a norma internacional ISO/IEC/IEEE 12207:2017(E) (ISO/IEC/IEEE, 2017, p. 82) define a finalidade do processo de verificação como "[…] prover evidência objetiva que o sistema ou elemento do sistema atende completamente seus requisitos e características especificados".

A mesma norma define que:

> […] o processo de verificação identifica anomalias (erros, defeitos e falhas) em qualquer item de informação (requisitos de sistema/*software* ou descrição de arquitetura), elementos implementados, ou processos de ciclo de vida usando métodos, técnicas, padrões e regras apropriados (ISO/IEC/IEEE, 2017, p. 76).

Existem diversas formas de fazer a verificação dos requisitos. Uma delas é a **revisão por pares**, que nada mais é do que um par realizando a revisão sobre o trabalho do colega, com o objetivo de identificar inconsistências ou anomalias. Um par é um profissional de *expertise* equivalente ou superior a do(s) autor(es) do artefato, que pode contribuir tecnicamente, fazendo uma avaliação objetiva do artefato.

Artefato é o resultado da realização de uma atividade, podendo ser um documento, um diagrama ou o próprio código. Revisões por pares podem ser aplicadas a qualquer artefato do projeto. Vamos aqui discutir a sua aplicação para a verificação dos artefatos relacionados às etapas de requisitos, mas o raciocínio utilizado se aplica a qualquer outro tipo de artefato do ciclo de vida de desenvolvimento, incluindo o código.

As revisões por pares podem acontecer no formato de **inspeções, *walk-throughs* estruturados**, **auditorias** ou outras formas de revisão. Sua aplicação vai depender do grau de formalismo desejado para a revisão. Inspeções são consideradas revisões mais formais e têm um formato próprio para acontecer.

As revisões também podem ser feitas seguindo diferentes abordagens. Podem ser baseadas em ***checklists***, em **cenários** de execução, com foco na **perspectiva** de algum usuário, com foco em uma **funcionalidade** específica ou ser feita até mesmo de forma *ad hoc*.



Saiba mais

Um dos requisitos não funcionais importantes referentes à qualidade interna de um produto de *software* é a **verificabilidade**. Verificabilidade se refere a "[...] quão bem os componentes de *software* ou o produto integrado podem ser avaliados para demonstrar se as funções do sistema funcionam conforme o esperado" (WIEGERS; BEATTY, 2013, p. 286).

Para identificar esses requisitos, você pode utilizar como dica as seguintes perguntas:

- Como nós poderemos confirmar que cálculos específicos estão fornecendo os resultados esperados?
- Existe alguma parte do sistema que não leva a resultados determinísticos, de forma que poderia ser difícil de determinar se está funcionando corretamente?
- É possível ter conjuntos de dados de teste que tenham uma alta probabilidade de revelar quaisquer erros nos requisitos e sua implementação?
- Que relatórios de referência ou outras saídas podem ser utilizadas para verificar se o sistema está produzindo os resultados corretamente?

Artefatos de requisitos

Se estamos focando em revisões dos requisitos, a pergunta é: que artefatos devem ser revisados? Os requisitos, uma vez elicitados, podem ser representados de diversas formas, dependendo do contexto do projeto, da organização desenvolvedora, das exigências do contratante, da experiência da equipe desenvolvedora e outros.

Vamos considerar neste capítulo duas formas de especificar requisitos: a abordagem orientada a casos de uso e a abordagem orientada e histórias de usuário. Além disso, vamos considerar também a lista de requisitos, que pode ser utilizada nas duas abordagens. Dessa forma, os seguintes artefatos serão considerados neste capítulo:

- Lista de requisitos funcionais e não funcionais.
- Casos de uso:
  - Diagrama de casos de uso.
  - Especificação de casos de uso.
- Histórias de usuário:
  - Cartão da história de usuário.
  - Critérios de aceitação das histórias de usuário.

Checklist para revisão de artefatos de requisitos

Um *checklist* pode ser considerado como uma lista contendo pontos de atenção para apoiar os revisores. Geralmente, ela é constituída dos itens que anteriormente já foram considerados problemáticos e que a equipe deseja evitar. O ideal é que cada artefato tenha seu *checklist* com itens próprios.

Vamos começar com alguns critérios que podem ser usados para analisar a **lista de requisitos**, que pode conter tanto os requisitos funcionais quanto os requisitos não funcionais. O Quadro 1 apresenta uma sugestão para este *checklist*. Itens identificados como "não" representam pontos de atenção que deverão ser priorizados e tratados pelo analista de requisitos. É sugerido que o revisor complemente as informações dos itens assinalados como "não", de modo a facilitar as correções.

Quadro 1. *Checklist* para a lista de requisitos

Atributo	Definição	Sim	Não	Não se aplica
Completo	O requisito está especificado de forma completa e que possibilita que o desenvolvedor o implemente?			
Correto	O requisito reflete o que o usuário, cliente ou seus representantes desejam?			
Único	O requisito descreve uma única capacidade, característica, restrição ou atributo de qualidade?			

(Continua)

(Continuação)

Quadro 1. Checklistt para a lista de requisitos

Atributo	Definição	Sim	Não	Não se aplica
Viável	O requisito é viável técnica e financeiramente de ser implementado, de acordo com as restrições do projeto?			
Necessário	O requisito tem um motivo de existir, que é representado pelo seu relacionamento a uma fonte de informação e a um objetivo de negócio?			
Priorizado	O requisito tem uma prioridade atribuída para que possa ser alocado a uma versão do *software*?			
Não ambíguo	O requisito não contém ambiguidades que levem os *stakeholders* a interpretá-lo de forma diferente?			
Verificável	O requisito é possível de ser verificado posteriormente quanto à sua implementação?			
Conforme	O requisito está em conformidade com os padrões de especificação estabelecidos, se houver?			

Fonte: Adaptado de Wiegers e Beatty (2013) e ISO/IEC/IEEE (2018).

O diagrama de casos de uso é um diagrama que ajuda o analista de requisitos a se comunicar com os usuários. Embora seja composto por elementos simples, o diagrama tem um grande poder de comunicação. O Quadro 2 apresenta uma sugestão de *checklist* para verificação do **diagrama de casos de uso**. Itens identificados como "não" representam pontos de atenção que deverão ser priorizados e tratados pelo analista de requisitos.

Quadro 2. *Checklist* para o diagrama de casos de uso

Atributo	Definição	Sim	Não	Não se aplica
Atores (escopo)	Todos os atores estão representados no diagrama?			
Atores (formato)	Todos os atores estão representados por um boneco palito e um texto com a identificação do ator?			
Relacionamentos de generalização entre atores (representação)	O relacionamento de generalização entre os atores é representado por uma seta vazada que aponta do ator filho para o ator pai?			
Relacionamentos de generalização entre atores (significado)	O relacionamento de generalização expressa corretamente a intenção de herança entre os atores envolvidos?			
Casos de uso (representação)	Todos os casos de uso são representados por elipses e identificados por meio de um verbo no infinitivo seguido de um complemento?			
Casos de uso (escopo)	Todos os casos de uso que representam as funcionalidades que os *stakeholders* desejam estão representados?			
Relacionamentos dos casos de uso com atores	Todos os casos de uso estão ligados a pelo menos um ator?			

(Continua)

(Continuação)

Quadro 2. *Checklist* para o diagrama de casos de uso

Atributo	Definição	Sim	Não	Não se aplica
Relacionamentos de <*include*> entre casos de uso	Os relacionamentos de <*include*> estão representados por setas pontilhadas que partem do caso de uso principal para o caso de uso incluído?			
Relacionamentos de <*extend*> entre casos de uso	Os relacionamentos de <*extend*> estão representados por setas pontilhadas que partem do caso de uso que estende para o caso de uso principal?			
Relacionamentos de generalização entre casos de uso (representação)	O relacionamento de generalização entre os casos de uso é representado por uma seta vazada que aponta do caso de uso filho para o caso de uso pai?			
Relacionamentos de generalização entre casos de uso (significado)	O relacionamento de generalização expressa corretamente a intenção de herança entre os casos de uso envolvidos?			

O diagrama de casos de uso sozinho não é suficiente como especificação de requisitos funcionais, portanto é necessário complementar com um documento de especificação de casos de uso (que pode também ser feito dentro de uma ferramenta de modelagem).

O Quadro 3 apresenta uma sugestão de *checklist* para verificação das **especificações de casos de uso**. Alguns itens se referem à especificação e outros se referem à consistência entre o diagrama de casos de uso e as suas especificações correspondentes. Itens identificados como "não" representam pontos de atenção que deverão ser priorizados e tratados pelo analista de requisitos.

Quadro 3. *Checklist* para a especificação de casos de uso

Atributo	Definição	Sim	Não	Não se aplica
Identificação do caso de uso	O caso de uso tem um identificador único e um texto contendo um verbo no infinitivo e um complemento?			
Ator principal	O ator principal está identificado corretamente?			
Ator secundário	O ator secundário (se houver) está identificado corretamente e participa do caso de uso juntamente com o ator principal?			
Pré-condição	As pré-condições (se houver) estão descritas corretamente?			
Fluxo principal	Todos os passos do fluxo principal estão numerados sequencialmente e descrevem claramente o diálogo entre o ator e o sistema?			
Fluxos alternativos (descrição)	Todos os fluxos alternativos (se houver) estão numerados sequencialmente e descrevem claramente o diálogo entre o ator e o sistema?			
Fluxos alternativos (retorno)	Todos os fluxos alternativos (se houver) definem qual é o próximo passo a ser executado ao seu término?			

(Continua)

(Continuação)

Quadro 3. *Checklist* para a especificação de casos de uso

Atributo	Definição	Sim	Não	Não se aplica
Fluxos alternativos (chamadas)	Para todos os fluxos alternativos (se houver) estão identificados os pontos de chamada no fluxo básico?			
Fluxos de exceção (descrição)	Todos os fluxos de exceção (se houver) estão numerados sequencialmente e descrevem claramente o diálogo entre o ator e o sistema em uma condição de exceção?			
Fluxos de exceção (retorno)	Todos os fluxos de exceção definem qual é o próximo passo a ser executado ao seu término?			
Fluxos de exceção (chamadas)	Para todos os fluxos de exceção estão identificados os pontos de chamada, seja no fluxo básico, seja nos fluxos alternativos?			
Pós--condições	As pós-condições (se houver) estão descritas corretamente?			

Quando a equipe de desenvolvimento utiliza métodos ágeis, é comum que os requisitos funcionais sejam expressos como **histórias de usuário**. As histórias de usuário são compostas por cartões, com uma declaração da necessidade do usuário e por critérios de aceitação que normalmente detalham a história ou especificam os requisitos não funcionais. O responsável pelos itens que compõem o *product backlog*, e que irão gerar as histórias, é o *product owner* (PO). Cabe a ele representar os interesses do cliente ou do usuário junto à equipe de desenvolvimento. As histórias de usuário geralmente são escritas de forma conjunta, entre o PO e a equipe de desenvolvimento.

A principal característica dos times ágeis é a comunicação. Os integrantes são estimulados a se reunir em *workshops* de esclarecimento de histórias e seguem a máxima dos 3 Cs: cartão + conversa + confirmação (critérios de aceitação). Portanto, é comum que as revisões já aconteçam ao longo do desenvolvimento das próprias histórias, durante esses *workshops* (PATTON, 2014).

No entanto, nem todos os ambientes ágeis funcionam tão bem. É bastante comum que as histórias de usuário sejam, na verdade, chamados que chegaram pelo Help Desk, foram sendo armazenados em um *backlog* de solicitações de mudança e foram alocados à *sprint* sem antes terem passado por uma conversa para aprofundar o seu entendimento. Nesses casos, a aplicação de um *checklist* pode apoiar a equipe na melhoria da qualidade da compreensão dessas histórias.

O Quadro 4 apresenta uma sugestão para o *checklist* de histórias de usuário. Note que esses critérios são muito similares aos critérios da lista de requisitos. É sugerido que o revisor complemente as informações dos itens assinalados como "não", de modo a facilitar as correções a serem realizadas pelo autor do artefato.

Quadro 4. *Checklist* para histórias de usuário

Atributo	Definição	Sim	Não	Não se aplica
Formato	A história de usuário está escrita no padrão: "Como (nome do papel), eu quero (…) de modo que (…)"?			
Completa	A história de usuário está especificada de forma completa e que possibilita que o desenvolvedor o implemente?			
Correta	A história de usuário reflete o que o usuário, cliente ou seus representantes desejam?			
Única	A história de usuário descreve uma única capacidade, característica, restrição ou atributo de qualidade?			

(Continua)

(Continuação)

Quadro 4. *Checklist* para histórias de usuário

Atributo	Definição	Sim	Não	Não se aplica
Viável	A história de usuário é viável técnica e financeiramente de ser implementada, de acordo com as restrições do projeto ou da *sprint*?			
Neces-sária	A história de usuário tem um motivo de existir, que é repre-sentado pelo seu relaciona-mento a uma fonte de informa-ção e a um objetivo de negócio?			
Priorizada	A história de usuário tem uma prioridade atribuída para que possa ser alocada a uma *sprint?*			
Não ambígua	A história de usuário não contém ambiguidades que levem os *stakeholders* a interpretá-la de forma diferente?			
Verificável	A história de usuário é possível de ser verificada posteriormente quanto à sua implementação?			

Melhoria da qualidade das revisões

As revisões por pares são momentos preciosos para promover a melhoria na qualidade das especificações de requisitos. Para que as revisões sejam proveitosas e se ganhe em produtividade, Wiegers (2006) apresenta as dicas a seguir.

- **Eduque os revisores:** explique para os revisores o que é esperado deles e qual será o processo usado na revisão. Aprender a dar *feedback* faz parte desse aprendizado. O foco da revisão deve estar sobre o artefato, e não sobre a pessoa.
- **Não sobrecarregue os revisores:** não espere que o documento esteja 100% completo. Ele pode ir sendo revisado aos poucos. É mais fácil conseguir 30 minutos de uma pessoa do que uma tarde toda.

- **Construa parcerias colaborativas com os representantes dos usuários e com outros membros da equipe do projeto:** a voz do cliente é muito importante na revisão dos requisitos, embora esta seja uma atividade de validação e não de verificação. Envolva o usuário ou representantes do usuário.
- **Convide os revisores certos:** envolva desenvolvedores e testadores também no processo de revisão, pois eles podem ter *insights* que os analistas de requisitos não têm. São visões diferentes que contribuem para a revisão. Eles podem usar a técnica de *perspective based reading*, ou leitura baseada em perspectiva (é uma leitura orientada sob a ótica específica daquele papel).
- **Faça os revisores analisarem os entregáveis apropriados:** nem todos os revisores terão que avaliar todos os artefatos de requisitos. Encaminhe o artefato para o revisor que mais pode contribuir com aquele artefato especificamente.
- **Projete para facilitar a revisão:** represente os requisitos de forma a facilitar a revisão.
- **Inspecione todos os entregáveis de requisitos:** embora as revisões informais sejam úteis, as inspeções formais são mais profundas e descobrem mais detalhes, uma vez que fazem com que os inspetores realmente analisem os artefatos.
- **Enfatize encontrar os maiores erros:** os requisitos faltantes são os mais difíceis de serem detectados. Invista tempo para este tipo de requisito.

3 Casos de teste de requisitos de *software*

A técnica mais utilizada para realizar a verificação de artefatos de desenvolvimento de *software* é o teste. Testes de *software* são utilizados para avaliar o código implementado, seja ele no formato de um componente isolado, seja no formato de um produto integrado.

Não é nosso objetivo neste capítulo o aprofundamento nas diversas técnicas de teste de *software*, mas apresentar os casos de teste como forma de refinamento e verificação de requisitos, tanto em ambientes tradicionais quanto em ambientes ágeis.

Testes em ambientes tradicionais

Geralmente, os casos de teste são escritos como apoio para que a equipe de testes possa realizar o seu trabalho. Existe uma variedade de formatos de escrita de casos de teste, que vão desde *templates* no formato de planilhas eletrônicas até ferramentas automatizadas, com robôs programados para executar os testes.

Independentemente da forma como o teste seja implementado, é importante compreender o que está envolvido no planejamento dos testes. Se voltarmos à Figura 2, veremos que existe uma etapa de testes representada logo após a codificação. Esse é um exemplo apenas didático e simplificado. Na realidade, os testes são realizados desde a fase de codificação, como forma do próprio desenvolvedor avaliar o seu trabalho. Em seguida, o produto segue para a equipe de testes, quando a empresa tem uma. E, finalmente, passa para o ambiente de homologação, onde iniciam os testes que envolvem o usuário ou um representante do usuário. Nesse caso, passamos a chamar de validação, e não mais de verificação, conforme as definições que já foram apresentadas anteriormente neste capítulo.



Fique atento

O grande benefício do desenvolvimento dos testes concomitantemente aos requisitos, ou logo após esses, é que os testes ajudam a esclarecer pontos obscuros dos requisitos. Refinamentos são possíveis sobre os requisitos, uma vez que se busca as formas para testá-lo. Visão similar foi incorporada pelos métodos ágeis, como se verá adiante.

Para a especificação dos casos de teste, o *template* apresentado no Quadro 5 pode ajudar. Inicialmente, deve ser identificada a origem do requisito. Neste caso, temos um exemplo usando casos de uso. Em seguida deve vir o identificador único do caso de teste, depois temos uma coluna onde são registrados os passos que o testador deve executar e qual é o resultado esperado.

Se tudo der certo e o sistema prover a resposta esperada, ele indica isto marcando um X na coluna ✓. Caso a execução não tenha ocorrido com sucesso, então ele marcará um X na coluna que aparece como ✗ na planilha. Nesse segundo caso, ou seja, o sistema não apresentou o resultado esperado, o testador deve reportar os detalhes para que o desenvolvedor possa corrigir.

Quadro 5. *Template* para casos de teste

ID do caso de uso	ID do caso de teste	Passos	Resultado esperado	✓	✗	N/A
<inserir o id do caso de uso>	<inserir o id do caso de teste>	<inserir os passos detalhados para a realização do caso de teste>	<inserir o resultado esperado ao final da execução do passo>			
EXEMPLO						
UC-CAD-001	CT-CAD-001	1 – Selecionar função Cadastrar Cliente	Tela de Cadastro de Cliente aberta	×		
		2 – Preencher CPF incorreto	Msg de erro exibida: "CPF inválido"		×	
		(…)				

Casos de teste são considerados ativos do processo de desenvolvimento e fazem parte do produto. A cada nova alteração em uma funcionalidade, os casos de teste precisam ser revisados para que continuem coerentes com as implementações. No caso de testes automatizados, é necessário que o código do teste seja refeito.

Testes em ambientes ágeis

Em ambientes ágeis de desenvolvimento de *software*, de acordo com Leffingwell (2011), é comum pensar de forma mais holística e sistemática, em que histórias (requisitos), implementação (código) e validação (testes de aceitação, testes unitários e outros) estejam juntos. Leffingwell (2011) apresenta uma matriz que descreve os tipos de teste em ambientes ágeis, ilustrada na Figura 3.

No quadrante Q1 estão os testes realizados pelo desenvolvedor, geralmente automatizados e em grande volume. São os **testes unitários e de componentes**. No quadrante Q2 estão os **testes funcionais**, que testam o funcionamento das histórias de usuário. Esses testes podem ser automatizados ou manuais. No quadrante Q3 se encontram os **testes de aceitação** ou testes de sistema; geralmente são testes manuais, que envolvem usuários e testadores em ambientes que simulam o ambiente real. No quadrante Q4 estão os **testes de qualidade** do sistema; geralmente envolvem o uso de ferramentas para os testes de desempenho e de carga.

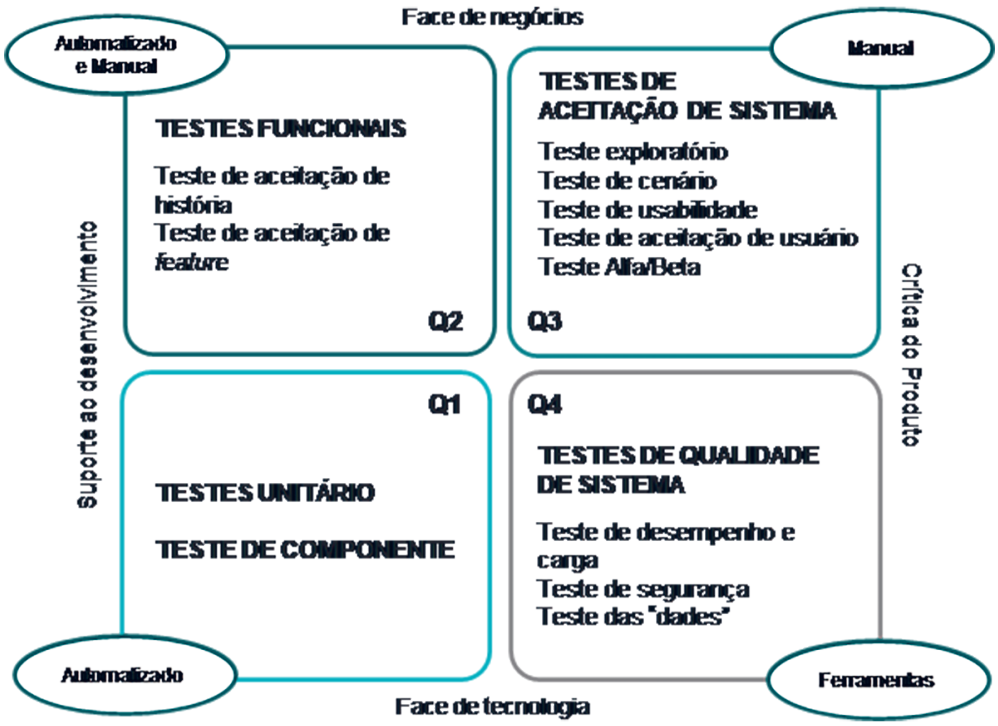


Figura 3. A matriz de testes ágeis.

Fonte: Adaptada de Leffingwell (2011).

Os **testes de aceitação de histórias**, representados no quadrante Q2 da Figura 3, são testes realizados pelos desenvolvedores. Trata-se de testes funcionais para garantir que as histórias de usuário entregam o que era pretendido. Geralmente, o time ágil passa grande parte do tempo definindo esses testes, pois esta é uma forma de refinar o comportamento esperado da história.

É comum, nestes ambientes, que a história seja uma afirmação curta, escrita no formato padronizado que expressa quem + o que + porquê. Para que ela possa ser adequadamente compreendida, os testes de aceitação da história são escritos e devem conter todos os detalhes para que o desenvolvedor possa implementá-la.



Exemplo

Considere a seguinte história de usuário (LEFFINGWEL, 2011):

Como cliente, eu sempre vejo o preco atual da energia refletido no meu portal e nos meus dispositivos de modo que eu sei que os meus custos de uso da energia são precisos e refletem qualquer mudanca de preco.

Os seguintes testes de aceitação da história serão derivados:

1. *Verificar que o preco atual é sempre usado e que os números calculados são exibidos corretamente no portal e em cada dispositivo (ver anexos).*
2. *Verificar que o preco e os números calculados são atualizados corretamente quando o preco muda.*
3. *Verificar que o campo preco atual em si é atualizado de acordo com a agenda.*
4. *Verificar as mensagens de informacão/erro quando houver erro na precificacão (ver mensagens de erro aprovadas no anexo)*



Saiba mais

Para se aprofundar nos temas relacionados a teste de *software*, o capítulo 22 do livro *Engenharia de Software: uma abordagem profissional*, de Pressman e Maxim (2016), que apresenta estratégias e teste de *software*, é uma leitura que vale a pena.

Referências

BOEING 737 Max Lion Air crash caused by series of failures. *BBC*, 2019. Disponível em: https://www.bbc.com/news/business-50177788. Acesso em: 29 abr. 2020.

BOURQUE, P.; FAIRLEY, R. *SWEBOK*: guide to the software engineering body of knowledge: version 3. USA: IEEE Computer Society, 2014. Disponível em: https://www.computer.org/education/bodies-of-knowledge/software-engineering. Acesso em: 29 abr. 2020.

CALLEAM CONSULTING. *Denver Airport baggage handling system case study*. [*S. l.*], 2008. Disponível em: http://calleam.com/WTPF/wp-content/uploads/articles/DIABaggage.pdf. Acesso em: 29 abr. 2020.

CMMI INSTITUTE. *CMMI model V2.0*. Pittsburgh: CMMI Institute, 2018.

CMMI PRDUCT TEAM. *CMMI® for Development, Version 1.3*. Hascom AFB: Software Engineering Institute, 2010. Disponível em: https://resources.sei.cmu.edu/asset_files/TechnicalReport/2010_005_001_15287.pdf. Acesso em: 29 abr. 2020.

HINKEL, J. N. *Ariane 5*: um erro numérico (overflow) levou à falha no primeiro lançamento. São José dos Campos, [201-?]. Disponível em: https://www.ime.uerj.br/~demoura/Especializ/Ariane/. Acesso em: 29 abr. 2020.

ISO/IEC/IEEE. *ISO/IEC/IEEE 12207:2017(E)*: systems and software engineering: software life cycle processes. Switzerland: ISO, 2017.

ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018*: systems and software engineering: life cycle processes: requirements engineering. Switzerland: ISO, 2018.

LEFFINGWELL, D. *Agile software requirements*: lean requirements practices for teams, programs, and enterprises. Upper Saddle River: Adisson-Wesley, 2011.

PATTON, J. *User story mapping*: discover the whole story, build the right product. Sebastopol: O´Reilly, 2014.

VAZQUEZ, C. E.; SIMÕES, G. S. *Engenharia de requisitos*: software orientado ao negócio. Rio de Janeiro: Brasport, 2016.

WIEGERS, K. E. *More about software requirements*: thorny issues and practical advice. 3. ed. Redmond: Microsoft Press, 2006.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMANN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Parte 4

Validação de Requisitos de Software

O conteúdo deste livro
é disponibilizado
por SAGAH.

SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Validação de requisitos de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Reconhecer a importância do envolvimento dos *stakeholders* na validação de requisitos de *software*.
- Projetar a validação de requisitos funcionais e não funcionais de projetos de *software*.
- Tratar os resultados da validação de requisitos de projetos de *software*.

Introdução

É inquestionável que os usuários têm se tornado cada vez mais exigentes com relação à qualidade dos produtos de *software* que utilizam, quer seja no ambiente profissional, quer seja na vida pessoal. A intensa exposição a uma grande variedade de soluções tecnológicas fez com que todos nós nos tornássemos mais críticos e menos tolerantes com as falhas decorrentes da tecnologia.

Temos experiências que são extremamente satisfatórias e outras, nem tanto. Já não temos mais paciência quando o aplicativo do banco nos diz que uma transação está temporariamente indisponível e que devemos tentar mais tarde, ou quando um *site* de compras *on-line* não consegue finalizar a transação com a operadora de cartão de crédito. Queremos uma interface agradável para o nosso gosto pessoal e simples de usar; queremos disponibilidade e tempo de resposta imediato. Queremos tudo isso porque já tivemos experiências desse tipo e desejamos que elas se repitam em todos os *softwares* com os quais temos contato.

No entanto, quem se dedica ao desenvolvimento de *software* sabe que atender a todas essas expectativas não é trivial e, muitas vezes, nem é possível. E os problemas começam nas primeiras fases, quando ainda estamos elicitando os requisitos. É comum surgirem primeiro os requisitos funcionais, que são os mais fáceis de identificar, e, posteriormente, os não funcionais, que muitas vezes ficam implícitos e invisíveis até que o usuário tenha contato com o produto.

Quanto mais cedo envolvermos no processo os diversos *stakeholders*, especialmente os usuários ou os seus representantes, maiores serão as chances de sucesso. Esse envolvimento começa com as atividades de elicitação, mas deve prosseguir com atividades constantes de validação, ou seja, de revisão crítica sobre aquilo que estamos construindo. E isso não pode ocorrer apenas na data final de entrega, pois, nesse momento, o estrago já terá sido feito e a insatisfação do usuário e o retrabalho serão inevitáveis.

Neste capítulo você vai ler sobre a importância do envolvimento dos *stakeholders* no processo de validação, desde os primeiros estágios do desenvolvimento. Vai também estudar como projetar a validação de requisitos funcionais e não funcionais de projetos de *software* e tratar os resultados provenientes da validação.

1 Importância dos *stakeholders* na validação de requisitos de *software*

Para compreendermos o processo de validação de requisitos, é importante relembrarmos as atividades envolvidas na engenharia de requisitos, ilustradas na Figura 1 e descritas no SWEBOK (*Software Engineering Body of Knowledge v3*):

> A área de conhecimento de Requisitos de Software (KA) preocupa-se com a obtenção, análise, especificação e validação de requisitos de *software* bem como o gerenciamento de requisitos durante todo o ciclo de vida do produto de *software* (BOURQUE; FAIRLEY, 2014, documento *on-line*, tradução nossa).

Considere o semicírculo mais externo da Figura 1 como sendo as atividades da engenharia de requisitos e, o semicírculo interno, o desenvolvimento em si, ou seja, a implementação. Transversal a todo o processo está o gerenciamento de requisitos.

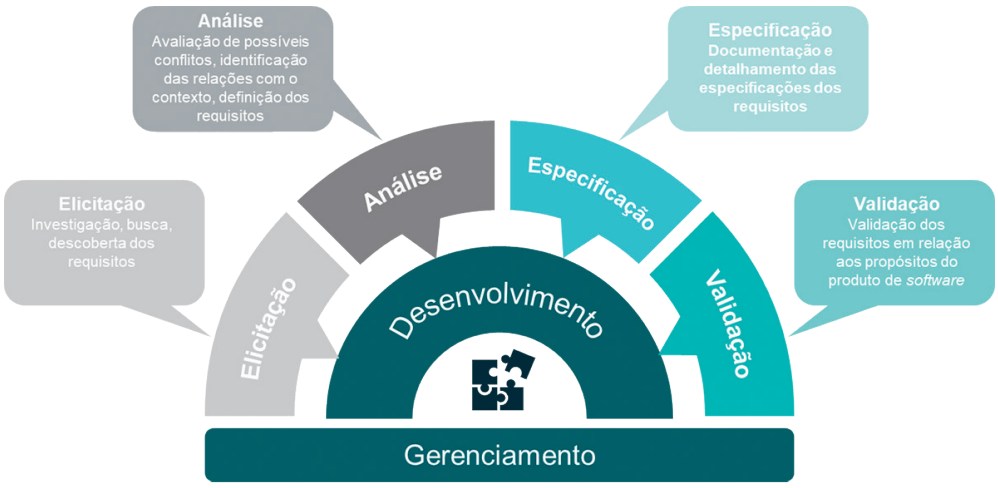


Figura 1. Etapas da engenharia de requisitos.

A disposição em forma de semicírculo é para destacar que não se trata de um processo cascata, no qual uma fase sucede a outra, mas, sim, de um processo iterativo e incremental, permitindo que as interações com o usuário ocorram em todos os momentos. Isso significa que existe uma **atividade** de elicitação de requisitos, e não uma fase de elicitação. O mesmo vale para a análise, a especificação e a validação — lembrando que as atividades de validação referenciadas no SWEBOK também englobam as atividades de verificação.



Fique atento

Verificação e validação são termos diferentes! De acordo com a norma internacional ISO/IEC/IEEE 12207 (ISO/IEC/IEEE, 2017, p. 10–11, tradução nossa):

- **Validação** é "a confirmação, por meio do fornecimento de evidência objetiva, que o requisito foi atendido para um uso ou aplicação pretendidos específicos".
- **Verificação** é a "confirmação, por meio do fornecimento de evidência objetiva, que os requisitos especificados foram atendidos".

Ainda, de acordo com o modelo de maturidade CMMI-DEV 2.0 (CMMI INSTITUTE, 2018, p. 525, tradução nossa), a diferença entre os termos é a seguinte:

- **Validação** "demonstra que a solução vai atender seu uso pretendido no ambiente alvo, isto é, 'você está construindo a coisa certa'".
- **Verificação** "endereça que o produto de trabalho ou a solução refletem adequadamente os requisitos especificados, isto é, 'você está construindo certo'".

Portanto, devemos lembrar que **verificação** analisa se o produto foi construído certo, ou seja, de forma correta; e que **validação** analisa se foi construído o produto certo, ou seja, aquele que os *stakeholders* desejavam (CMMI INSTITUTE, 2018).

O CMMI Institute (2018) define que validar os requisitos significa assegurar que a solução que será desenvolvida vai se comportar conforme o esperado no ambiente-alvo. Isso é importante para evitar os custos com retrabalho e para aumentar a satisfação do usuário ao receber uma solução que, efetivamente, atende às suas necessidades.

Neste capítulo vamos nos concentrar nas atividades relacionadas à validação de requisitos. Atividades de verificação de requisitos não serão aqui tratadas.

Conceitos de validação de requisitos

O que vem à sua mente quando se fala em validação de requisitos? É provável que você pense que validar seja o usuário testar o produto ou incremento do produto que está sendo liberado para identificar se ele atende às suas necessidades. Embora teste de *software* seja uma das técnicas de validação, ela não é a única. Podemos realizar atividades de validação desde muito cedo no ciclo de vida, mesmo antes de termos algum código implementado.

Embora nas fases iniciais de desenvolvimento os testes não possam ser executados sobre o produto rodando, porque ele ainda não existe, testes conceituais podem identificar problemas nos requisitos logo no início, revelando erros, ambiguidades e omissões (WIEGERS; BEATTY, 2013).



Saiba mais

De acordo com o modelo CMMI-DEV 2.0 (CMMI INSTITUTE, 2018), a **verificação** e a **validação** na engenharia de *software* envolvem simulação, estudos de rastreabilidade, revisões ou auditorias funcionais, revisões ou auditorias físicas, revisões por pares, demonstrações, protótipos, revisões formais, testes de módulo, regressão e integração de sistema.

As atividades de validação de requisitos são realizadas para identificar que (WIEGERS; BEATTY, 2013, tradução nossa):

- os requisitos de *software* descrevem, de forma precisa, as capacidades e propriedades do sistema que vão satisfazer às diversas necessidades dos *stakeholders*;
- os requisitos de *software* estão corretamente derivados dos requisitos de negócios, requisitos de sistema, regras de negócio e outras fontes;
- os requisitos estão completos, viáveis e verificáveis;
- todos os requisitos são necessários e o conjunto completo dos requisitos é suficiente para atender aos objetivos de negócios;
- todas as representações dos requisitos são consistentes umas com as outras;
- os requisitos proveem uma base adequada para prosseguir para o projeto (*design*) e a construção.

Quanto mais cedo no ciclo de vida identificarmos os defeitos nos requisitos, menos custoso será para corrigir. Defeitos que são inseridos nas fases iniciais do desenvolvimento vão se propagando em termos de custos adicionais no desenvolvimento de *software*. Se um requisito for compreendido de forma incorreta, ele resultará em desdobramentos também incorretos, como os requisitos detalhados, as especificações de requisitos, a codificação, os casos de teste e, por fim, o produto integrado e entregue ao cliente. A cada etapa que ele permanece não descoberto, novos custos são adicionados.

A Figura 2 ilustra um ciclo genérico de desenvolvimento, no qual os defeitos de requisitos estão representados pelas bolinhas no início do cone. Como você pode observar, ao incluir as atividades de validação ao longo do ciclo de vida (pontos 1, 2, etc.), esses defeitos tendem a diminuir, prevenindo os custos com retrabalho e a insatisfação do usuário. Se essas atividades não tivessem sido realizadas, é possível que a quantidade de defeitos inicial estivesse ainda presente no momento da homologação. Poderia ser ainda que ela tivesse se ampliado, uma vez que um defeito em um requisito pode implicar em defeitos em vários outros requisitos que dele dependem.

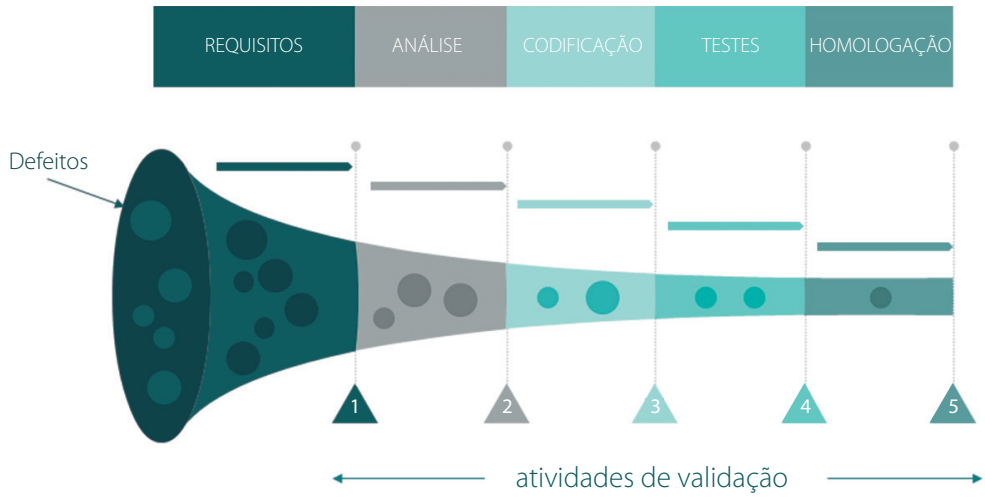


Figura 2. Defeitos inseridos e removidos ao longo do ciclo de desenvolvimento.

Embora a Figura 2 possa sugerir um ciclo cascata, novamente não é essa a nossa intenção. O que queremos representar aqui é que um requisito compreendido de forma errada será levado para as demais atividades e será implementado também de forma errada. Quanto mais tempo esse defeito permanecer escondido, mais custos de manutenção serão incorridos no futuro. Se o único momento em que o usuário vier a avaliar esse requisito for na hora da homologação, então o requisito terá que voltar para as origens, onde será novamente analisado, implementado e testado, gerando custos adicionais provenientes do retrabalho e inicialmente não previstos no projeto.

Quem deverá fazer a validação dos requisitos?

Independentemente da forma como se opte por realizar as atividades de validação, é inquestionável a necessidade de envolvimento do usuário desde o início. Compreender as classes de usuários é o primeiro passo para identificar as fontes de informação que serão a base para a elicitação de requisitos. E por que estamos falando de elicitação de requisitos em um capítulo que foca a validação de requisitos? Porque é na elicitação que tudo começa. Quando não envolvemos, de forma adequada e contínua, todos os *stakeholders* do projeto desde o início, a chance de construirmos o produto correto será muito pequena, ao passo que a chance de termos custos elevados de retrabalho e altos índices de insatisfação será grande.

Aquelas classes de usuários que foram identificadas na etapa de elicitação serão agora utilizadas para a validação dos requisitos. Geralmente, essas classes de usuários são as que terão alguma interface com o negócio que está sendo tratado pelo *software*. Quando o setor for muito afetado por leis ou regulamentações que precisam ser atendidas, uma classe de usuários deverá poder validar os requisitos de ordem legal.

Para apoiar a identificação dos *stakeholders* externos e internos que farão a validação, vamos relembrar no Quadro 1 as questões que Leffingwell (2011) recomenda que sejam feitas para identificá-los.

Quadro 1. Questões de apoio para identificar fontes de informação

***Stakeholders* externos**	***Stakeholders* internos**
Quem utilizará diretamente o sistema? Quem utilizará os resultados daqueles que usam o sistema? Quem será responsável por dar suporte ao sistema? Com quais outros sistemas o nosso sistema irá interagir? Que interfaces devem ser fornecidas? Quem pode fornecer orientações sobre as funcionalidades e os itens de qualidade do sistema (usabilidade, confiabilidade, desempenho e manutenibilidade)?	Quem precisa ser consultado sobre o escopo do projeto? Quem contribuiu para o orçamento e o cronograma? Quem gerencia o relacionamento de negócios entre as equipes e o cliente? Quem irá determinar como e quando o sistema deve ser entregue para os clientes? Quem pode afetar ou apoiar politicamente o projeto? Que parceiros dependem do sistema? Quem se preocupa com o processo que será usado no desenvolvimento do sistema?

Fonte: Adaptado de Leffingwell (2011).

Neste momento podemos ainda revisitar as personas que foram definidas para representar as classes de usuários. Leffingwell (2011) define uma **persona primária** como alguém que interage com o *software* e que necessita de uma interface projetada especificamente para ela. Uma **persona secundária** é um usuário que utiliza o *software* com uma interface projetada para outro tipo de usuário. Persona é um representante hipotético, genérico, de uma classe de usuários (WIEGERS; BEATTY, 2013). No momento da validação, é necessário identificar quem fará a validação como representantes das personas.

Validação de produtos de *software* × validação de serviços

Como temos dito, a validação vai depender do tipo de projeto. Quando estamos falando de projetos de *software* que habilitam negócios inovadores, como aqueles desenvolvidos pelas *startups*, por exemplo, é possível que a percepção sobre qualidade do produto de *software* se confunda com a percepção sobre a qualidade do serviço prestado. Isso é bastante comum em produtos que se confundem com serviços.

Vamos tomar como exemplo os aplicativos para chamar um transporte (como o Uber, o 99 ou similares). Se você pedir um carro e o aplicativo não conseguir localizar um motorista nas proximidades, ele vai indicar um motorista mais distante, que vai demorar mais tempo para chegar. Você se sentirá insatisfeito, mas esse não é um problema proveniente do produto de *software*, mas sim do serviço que é prestado por meio do produto de *software*. Você pode optar por desinstalar aquele aplicativo e utilizar outro, mas o motivo do abandono não terá sido o aplicativo em si, mas o serviço.

Por outro lado, considerando ainda o caso anterior, se você não conseguir localizar onde você adiciona uma parada intermediária no seu trajeto, então esse sim será um problema de usabilidade do produto de *software*, que pode afetar a forma como você percebe a qualidade do serviço como um todo. Esse também pode ser um motivo para você desinstalar o aplicativo e passar a utilizar outro, mas, nesse caso, o motivo terá sido o *software*, e não o serviço.

Uma forma menos evidente de diferença entre o defeito do *software* e do serviço seria, por exemplo, o oferecimento de opções de pagamento diferentes. Se a opção de pagamento no aplicativo de transporte for apenas via cartão de crédito isso pode ser um problema de *software* (não foi implementada a opção do pagamento em dinheiro, embora tivesse sido solicitada pelo demandante do *software*) ou pode ser um problema do serviço (o negócio não prevê que seja feito o pagamento em dinheiro, permitindo apenas receber via cartão de crédito). Conseguiu perceber a diferença?

Validar produtos de *software*, portanto, não pode ser confundido com validar o serviço que ele presta. Essa linha é tênue e nem sempre é fácil identificar claramente essa separação. Por isso os procedimentos de validação devem ser cuidadosamente planejados para que possam ser realizados de maneira eficaz na descoberta antecipada de defeitos no produto.

2 Planejamento da validação de requisitos

Conforme apresentado anteriormente, a norma internacional ISO/IEC/IEEE 12207:2017(E) (ISO/IEC/IEEE, 2017, p. 89, tradução nossa), define a finalidade do processo de validação da seguinte maneira: "[…] fornecer evidência objetiva de que o sistema, quando em uso, cumpre seus objetivos de negócio ou de missão e os requisitos dos *stakeholders*, alcançando seu uso pretendido no seu ambiente operacional pretendido". A mesma norma complementa que "[…] esse processo provê as informações necessárias para que as anomalias identificadas possam ser resolvidas pelos processos técnicos apropriados onde essa anomalia foi criada" (ISO/IEC/IEEE, 2017, p. 89, tradução nossa).

Para sistemas de *software*, a norma estabelece que a finalidade do processo de validação é a seguinte:

- confirmar que os requisitos para um uso específico de um produto de trabalho foram atendidos (normalmente chamado de validação de *software*);
- atingir a confiança (especialmente de um adquirente ou um cliente) que o produto entregue atende às necessidades dos *stakeholders* (normalmente chamado de teste de aceitação de *software*).

Veja que nesse último item aparece a figura do adquirente ou cliente. Essa é uma observação importante, pois nem sempre o cliente que adquire o produto é necessariamente o usuário do produto. Nesse caso o cliente representa os interesses do usuário do produto.

Processo de validação

O processo de validação pode ser compreendido como um conjunto de três atividades, conforme ilustra a Figura 3. A primeira atividade é **preparar para a validação**, ou seja, realizar o planejamento de como será conduzida a validação. em seguida, temos a atividade de **realizar a validação**, que, como o próprio nome sugere, refere-se às atividades de execução da validação e de registro dos seus resultados. por fim, temos a atividade de **gerenciar os resultados da validação**, na qual os resultados da validação são analisados para que planos de ação possam ser executados.



Figura 3. Atividades do processo de validação.
Fonte: Adaptada de ISO/IEC/IEEE (2017).

Preparar a validação envolve tarefas específicas relacionadas à definição das estratégias que serão adotadas para a validação, como, por exemplo:

- que produtos de trabalho (artefatos) serão validados;
- quais são as restrições que podem existir para que a validação seja executada;
- quais itens são prioritários;
- que métodos e técnicas serão empregados na validação, bem como os respectivos critérios de aceitação;
- que sistemas ou ferramentas serão necessários para permitir a validação e como eles serão obtidos.

A preparação da validação é uma tarefa tão importante quanto a sua realização. Ao identificarmos a necessidade de validação de um artefato (documento ou o próprio código), é importante refletir sobre como acontecerá essa validação, quem será envolvido, que materiais serão necessários, qual o escopo da validação, que técnica será utilizada para validar. Para cada tipo de técnica, um tipo diferente de planejamento será necessário.

Quando a validação estiver concluída, é preciso **gerenciar os resultados**. Se o produto estiver de acordo com o esperado, basta formalizar o aceite e dar por encerrado o processo de validação, seguindo com o projeto/versão para as demais atividades. Caso anomalias tenham sido encontradas, elas deverão ser analisadas e as ações necessárias deverão ser identificadas, priorizadas e executadas. Nova rodada do processo de validação pode ser requerida, nesses casos. Discutiremos isso na próxima seção.



Fique atento

Embora estejamos nos referindo sempre ao termo **processo de validação**, vale aqui lembrar que não se trata de uma burocratização das atividades, mas de **conceitos que são importantes** quando realizamos a validação, independentemente do método de desenvolvimento que está sendo utilizado.

Em **ambientes ágeis de desenvolvimento**, onde é esperado que o cliente se faça presente de forma mais contínua, as atividades de validação podem ocorrer simultaneamente às atividades de elicitação de requisitos. Isso normalmente se dá por meio da discussão sobre as histórias de usuário e seus critérios de aceitação, que ocorrem de forma continuada ou em *workshops* de validação.

Como realizar a validação de requisitos

Os requisitos podem ser validados de diversas formas, desde a inspeção cuidadosa dos artefatos de requisitos até o teste do produto em relação aos requisitos inicialmente identificados e às necessidades dos usuários. O modelo de melhoria de processos de desenvolvimento CMMI-DEV v1.3 (CMMI PRODUCT TEAM, 2010) identifica as seguintes formas de validação:

- discussões com os usuários finais (em um contexto de inspeções formais, por exemplo);
- demonstrações de protótipos;
- demonstrações funcionais (por exemplo: sistema, unidades de *hardware*, *software*, documentação de serviços, interfaces de usuário);
- pilotos com materiais de treinamento;
- testes de produtos e componentes de produtos por usuários finais e outros *stakeholders* relevantes;
- entregas incrementais de produtos funcionais e potencialmente aceitáveis;
- análises de produtos e componentes de produtos (simulações, modelagens, análises de usuários).



Saiba mais

Quando as atividades de validação envolverem componentes de *hardware*, considere incluir modelagem para validar a forma, a adequação e a função dos projetos mecânicos; modelagem térmica; análises de manutenibilidade e confiabilidade; demonstrações ao longo do tempo; e simulações de projeto elétrico de componentes eletrônicos e mecânicos (CMMI PRODUCT TEAM, 2010, documento *on-line*).

Técnicas de validação

Existem diversas formas de fazer a validação dos requisitos. Vamos destacar aqui a **inspeção dos artefatos gerados** e o **teste do produto ou componente do produto**. Para cada uma delas, um planejamento será necessário. Se estivermos nos referindo a uma inspeção nos requisitos a ser realizada pelos *stakeholders*, então os elementos envolvidos serão os próprios requisitos e quem realizará a validação. Nesse caso, talvez tenhamos apenas que prever materiais de apoio, que podem incluir um computador para registro dos resultados ou notinhas adesivas tipo Post-it®.

No entanto, se estivermos falando de um **teste no produto ou componente do produto**, então um ambiente próprio precisará ser montado e configurado, o que envolve, possivelmente, *hardware*, *softwares* de apoio, bases de dados, etc. Normalmente, esses ambientes são chamados de ambientes de homologação, pois visam a reproduzir, com a maior fidelidade possível, o ambiente-alvo onde o *software* será executado quando estiver entregue.

A validação utilizando o conceito de revisões pode acontecer no formato de **inspeções formais**, ***walkthroughs* estruturados**, **auditorias** ou outras formas de revisão. Sua aplicação vai depender do grau de formalismo desejado para a validação. Inspeções são consideradas revisões mais formais e têm um formato próprio de funcionamento.

Neste momento, você deve estar se perguntando se isso não seria a mesma coisa que realizar uma verificação, uma vez que as técnicas são as mesmas. Embora as técnicas possam ser as mesmas, existe uma grande diferença: a validação pressupõe que o usuário, ou o seu representante, participe do processo. Lembre-se de que o foco da validação é garantir que estamos construindo o que o usuário deseja, ou seja, o produto certo. Vamos discutir um pouco mais sobre isso.

As revisões com foco na validação podem ser feitas seguindo diferentes abordagens. Elas até podem ser baseadas em ***checklists***, mas isso não será suficiente sob a ótica de quem utilizará o produto. Embora usar um *checklist* seja uma boa forma de apoiar uma verificação, ele não se mostra tão adequado para apoiar a validação.



Exemplo

Vamos tomar como exemplo um item que tipicamente está presente em um *checklist* de verificação de requisitos: "O requisito está especificado de forma completa e que possibilita que o desenvolvedor o implemente?". Esse é um item que não poderá ser avaliado pelo usuário, ele não sabe se o requisito está no nível adequado para o desenvolvedor implementar. Outro exemplo: "O requisito é viável técnica e financeiramente de ser implementado, de acordo com as restrições do projeto?". O usuário não terá elementos para julgar se ele é viável, o máximo que ele poderá contribuir nesse caso é dizer que um requisito é mais importante do que o outro, levando à uma priorização de requisitos em caso de recursos escassos.

Para o usuário, é mais eficiente que a validação utilize **cenários** de execução, com foco na **perspectiva** de um grupo específico de usuários ou com foco em uma **funcionalidade** específica, quando ela for mais complexa.

Caso você tenha tido alguma experiência com ambientes que usam métodos ágeis, já deve estar relacionando a validação com o **mapeamento de histórias de usuário**. E está correto! O mapeamento de uma história de usuário é uma forma de realizar a validação de uma solução, desde que o usuário ou o representante dele faça parte desse mapeamento.

A validação utilizando técnicas de **teste de *software*** envolve o planejamento da exposição do usuário aos requisitos implementados. Inspeções e cenários podem ser considerados como uma espécie de teste conceitual. No entanto, teste, no conceito mais estrito do termo, significa expor o usuário ao produto, ou parte do produto implementado, mesmo que na forma de uma versão preliminar ou de um protótipo.



Saiba mais

Em ambientes ágeis que utilizam a abordagem *lean startup*, existe um conceito muito forte que é a exposição do usuário, o mais cedo possível, a um produto mínimo viável, mais conhecido pela sua sigla **MVP**, que vem do inglês *minimun viable product*. Isso significa que você precisa compreender em profundidade a dor do usuário e apresentar uma pequena solução para que ele possa validar se aquilo vai atendê-lo.

Acompanhe um exemplo no livro de Jeff Patton, *User Story Mapping* (em inglês), que descreve a história de um sistema de acesso à biblioteca de uma faculdade nos Estados Unidos e como eles usaram os conceitos de *lean startup* para desenvolver uma solução mais eficiente. (PATTON, 2014).

Segundo Patton (2014), testar uma solução envolve mais do que buscar os defeitos, uma vez que um *software* sem defeitos não garante a satisfação do usuário. Para Patton (2014, p. 214, tradução nossa), é preciso que você "Coloque as suas soluções em frente às pessoas que irão comprar ou usar o seu produto. Não espere que elas sejam um sucesso da primeira vez. Faça iterações e melhore-as".

Testes em ambientes ágeis

É possível que você já tenha visto alguma vez a matriz que descreve os tipos de testes em ambientes ágeis proposta por Leffingwell (2011), ilustrada na Figura 4. Na realidade, embora o foco seja nos testes em ambientes ágeis, se observarmos mais detidamente, veremos que ela se aplica a qualquer ambiente, mesmo aqueles que não compartilham dos princípios ágeis.

Os quadrantes superiores, que apresentam a **face de negócios**, são os que se dedicam às atividades de validação, pois envolvem os testes de aceitação das funcionalidades e sistema. Isso pode ser observado no quadrante Q2 (Testes funcionais) e no quadrante Q3 (Teste de aceitação de sistema).

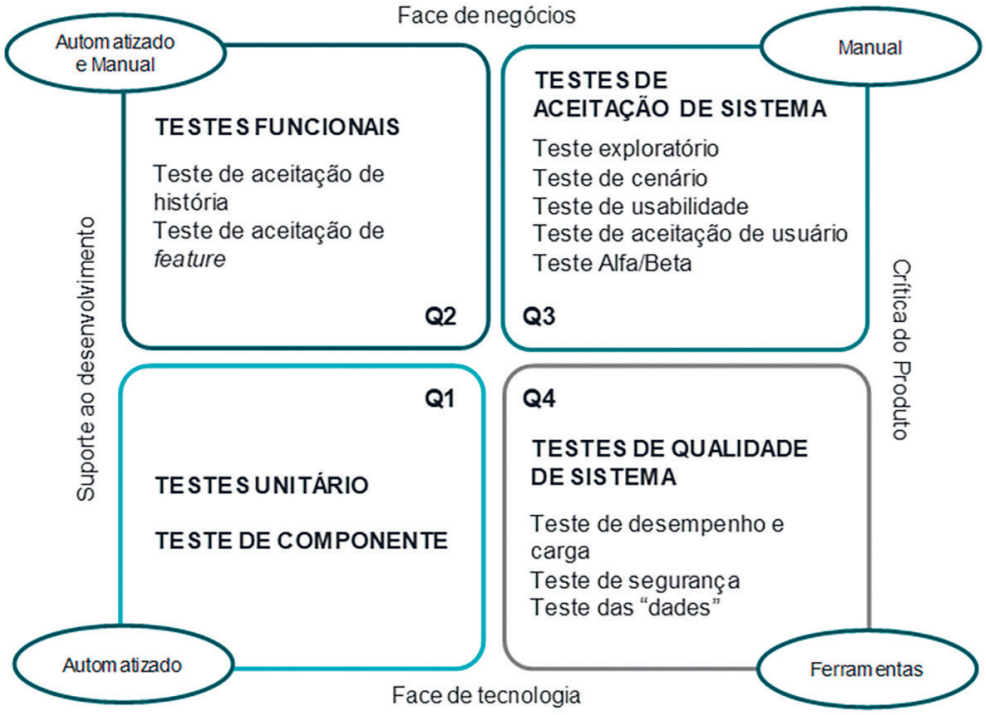


Figura 4. A matriz de testes ágeis.

Fonte: Adaptada de Leffingwell (2011).

O **teste de aceitação de histórias** e o **teste de aceitação de *features***, representados no quadrante Q2 da Figura 4, são testes realizados pelos desenvolvedores com base nos critérios que são escritos como complemento às histórias de usuário.

Vale aqui relembrar que as histórias de usuário são descritas normalmente levando em consideração os 3 Cs: **C**artão, **C**onversa e **C**onfirmação (LEFFINGWELL, 2011). O **cartão** deve conter a declaração principal da história, que deve ter os seguintes elementos: quem + o que + porque, ou seja, quem é o papel que executará a função, o que ele precisa e por que ele precisa. A **conversa** deverá ser o elemento principal no relacionamento entre a equipe e os *stakeholders*, ela é a base da validação contínua dos requisitos. Por fim, a **confirmação**, que se refere aos critérios de aceitação que serão usados para validar as histórias de usuário.

Já o quadrante Q4 apresenta os **testes de aceitação de sistema**, que são tipicamente realizados pelos *stakeholders* e que constituem a validação dos requisitos no produto implementado e integrado. São eles:

- teste exploratório;
- teste de cenário;
- teste de usabilidade;
- teste de aceitação do usuário;
- teste alfa/beta.

Assim como os requisitos funcionais, também os requisitos não funcionais podem ser validados desde sua elicitação. A diferença é que é mais fácil para os *stakeholders* definirem os requisitos funcionais do que os requisitos não funcionais, especialmente em termos quantitativos. Por esse motivo, é importante que os procedimentos de elicitação para esses requisitos sejam baseado em *checklists*, que ajudam a não esquecer de algum item importante para o produto. Uma boa dica nesse caso é olhar para a família de normas ISO/IEC 25.000.



Saiba mais

Para se aprofundar nos temas relacionados a teste de *software*, uma leitura interessante é o Capítulo 22 do livro *Engenharia de Software: uma abordagem profissional*, de Pressman e Maxim (2016), que apresenta estratégias e teste de *software*.

3 Tratamento dos resultados da validação de requisitos

Quando uma rodada de validação de requisitos é encerrada, ela gera resultados que precisam ser analisados de forma conjunta entre os *stakeholders* do projeto e a equipe de desenvolvimento, de modo a gerar ações corretivas. Os resultados podem estar relacionados à identificação destes pontos:

- erros nos requisitos existentes;
- mudança nos requisitos existentes;
- novos requisitos;
- conflitos entre requisitos.

Um **erro em um requisito** ocorre quando ele foi identificado de forma incorreta pelo analista de requisitos ou quando foi escrito de forma incorreta. Isso pode ser decorrente de suposições que o analista de requisitos fez e que não estavam adequadas, ou pode ser que ele simplesmente tenha cometido alguma falha na hora da escrita. Independentemente da causa, o erro deve ser corrigido antes que possa seguir para a implementação. Quanto mais cedo o erro for detectado, menos oneroso ele será para o projeto.

No momento da validação, **mudanças nos requisitos existentes**, ou até mesmo **novos requisitos**, podem surgir, seja em decorrência do desdobramento de requisitos existentes, seja como consequência da melhor compreensão do produto pelos *stakeholders*. Em ambos os casos isso representa esforço adicional de desenvolvimento, assim como custos e recursos adicionais. Isso pode gerar um impacto no cronograma do projeto ou da versão, bem como no orçamento inicialmente estabelecido.

Outro aspecto que pode ser identificado durante a validação de requisitos é o conflito entre os requisitos. Isso pode ser decorrente de visões diferentes entre *stakeholders* ou pode ser proveniente de um único *stakeholder*. Conflitos denotam requisitos que são mutuamente exclusivos e que precisam ser tratados, pois não podem seguir dessa forma para a implementação.

Quanto mais cedo esses problemas forem detectados, mais fácil e mais barato será corrigi-los. Quando o cliente está distante, ou até mesmo ausente do desenvolvimento, a validação pode ser adiada para fases posteriores, quando o produto já estará construído, e então os custos para reparar esses problemas serão maiores.

Um plano de ação idealmente leva em consideração o seguinte:

- impacto da mudança sobre o requisito afetado;
- impacto da mudança sobre outros requisitos afetados relacionados;
- restrições de custo e prazo impostas ao projeto ou à versão/*sprint*;
- riscos da alteração;
- riscos da não alteração.

Como resultado, decisões deverão ser tomadas. Quando se tratar de um erro, ele terá, obrigatoriamente, que ser corrigido antes que prossiga para a implementação. No entanto, quando se tratar de uma melhoria ou de um novo requisito, pode ser que a opção seja pela não implementação ou pelo adiamento da alteração para versões posteriores do produto.

Em ambientes ágeis, é comum que o *product owner* seja o responsável por essa decisão, junto com o time de desenvolvimento. Em ambientes mais tradicionais essa decisão será tomada pelo cliente juntamente com o gerente de projetos, com o apoio do analista de requisitos.

Fique atento

Tão importante quanto identificar os problemas nos requisitos o mais cedo possível é identificar sua **causa raiz**. A identificação das causas vai permitir a adoção de ações preventivas, com vistas a evitar que o mesmo erro venha a se repetir no projeto corrente e em projetos futuros.

Quando a causa raiz não é identificada, é possível que, em algum momento posterior, os mesmos erros sejam repetidos. Ações decorrentes da identificação dessas causas incluem prover treinamento para a equipe de desenvolvimento, realizar melhorias no processo de engenharia de requisitos, adquirir ferramentas de apoio ao processo de engenharia de requisitos ou até mesmo substituir membros da equipe.

Tratamento de conflitos

Quando for observado o conflito entre as visões de dois ou mais *stakeholders*, então esse conflito deverá ser gerenciado, de acordo com Pohl e Rupp (2015), considerando as etapas ilustradas na Figura 5.



Figura 5. Fases da resolução de conflitos.

Conflitos podem surgir em qualquer etapa da engenharia de requisitos. Uma vez realizada a atividade de **identificação do conflito**, vem a etapa de **análise do conflito**. Um conflito pode surgir pelos seguintes motivos (POHL; RUPP, 2015, p. 105–106):

- **Conflito de dados:** quando existe divergência entre os dados fornecidos por *stakeholders* diferentes. Por exemplo, um *stakeholder* pode achar que o tempo de resposta adequado para determinada funcionalidade é de menos de 1 segundo, enquanto outro pode achar que 1 segundo é inviável tecnicamente.
- **Conflito de interesses:** quando existe uma diferença de expectativas entre dois ou mais *stakeholders*. Por exemplo, um acha que os custos do projeto devem ser mantidos no nível mínimo, enquanto outro acha que a qualidade deve ser a prioridade.
- **Conflito de valores:** quando existe uma diferença entre os valores pessoais dos *stakeholders*, que podem ser provenientes de culturas diferentes. Isso pode ocorrer, por exemplo, quando um dos *stakeholders* defende o uso de tecnologias *open source* enquanto outro defende o uso de tecnologias proprietárias.
- **Conflito de relacionamento:** quando existem personalidades fortes e antagônicas. Isso pode ocorrer quando dois *stakeholders*, geralmente de mesma posição hierárquica, conflitam sobre um requisito e um deles deseja impor a sua opinião sobre o outro.
- **Conflito estrutural:** quando existe diferença de perspectivas entre pessoas de níveis hierárquicos diferentes. Pode ser, por exemplo, que um nível hierárquico superior não ache que um subordinado tenha competência para especificar aquele tipo de requisito.

Dificilmente os conflitos são tão claramente classificáveis, pois é comum que um seja consequência, ou esteja atrelado ao outro. Por exemplo, um conflito de relacionamento pode ser decorrente de um conflito de valores.

Após a análise vem a **resolução do conflito**, que constitui uma das principais etapas da validação de requisitos. Se uma resolução deixar uma das partes insatisfeita, então, o comprometimento dessa parte com o projeto pode ser reduzido e até mesmo inviabilizado, ou seja, o *stakeholder* pode se tornar um detrator do projeto e, possivelmente, um obstáculo ao seu sucesso. Independentemente da natureza do conflito, é importante envolver os diversos *stakeholders* do projeto na busca por uma alternativa de resolução.

Existem diversas técnicas que podem ser utilizadas para resolver conflitos, entre elas (POHL; RUPP, 2015, p. 106–108):

- **Acordo:** acontece quando se busca uma solução de consenso entre as partes.
- **Comprometimento:** ocorre quando uma solução alternativa é buscada pelas partes sem conflito. A solução criada é nova e não as mesmas que haviam sido propostas.
- **Votação:** ocorre quando são apresentadas as soluções alternativas e é feita uma votação. Essa não é a melhor solução, pois haverá vencedores e perdedores e, quando isso ocorre, o lado perdedor pode se tornar um obstáculo.
- **Definição de variantes:** ocorre quando se define uma forma de parametrização no sistema, para acomodar visões diferentes de requisitos.
- **Decisão superior (*overruling*):** ocorre quando a decisão é tomada pelo nível hierárquico superior aos conflitantes. Essa também é uma situação que deve ser evitada, pois traz como consequência o sentimento de ter havido um ganhador e um perdedor.
- **Considere todos os fatores (CAF):** lista-se os diversos atributos do requisito que está em conflito e, após análise desses atributos, chega-se a uma decisão sobre o conflito.
- **Mais-menos-interessante:** características positivas e negativas de uma solução são listadas, de modo que elas possam ser comparadas. O termo interessante é usado para denotar quando uma solução parece ser interessante, mas precisa ser melhor investigada.
- **Matriz de decisão:** ocorre quando se lista as alternativas em conflito nas colunas e os critérios de seleção nas linhas. Cada atributo é pontuado para cada uma das alternativas e, no final, apura-se o que teve a maior pontuação.

Por fim, é realizada a **documentação do conflito**, na qual todos os conflitos e decisões são documentados. Isso é importante para poder manter o padrão de decisões de conflitos ao longo do projeto, bem como servir de fonte para consulta sobre as decisões estabelecidas.

Prevenção de problemas em requisitos

Ao identificar as causas dos problemas em requisitos, mudanças na forma de trabalho das pessoas devem ser introduzidas para a prevenção de sua futura repetição. No entanto, de acordo com Wiegers e Beatty (2013, tradução nossa), podem surgir barreiras para que mudanças nos processos de requisitos aconteçam. A maioria delas não é de ordem técnica, mas sim pessoal. Entre elas estão as seguintes:

- falta de reconhecimento dos problemas que as práticas atuais de requisitos causam;
- falta de tempo — todos já estão muito ocupados;
- pressão do mercado ou da gerência para entregar rapidamente;
- falta de comprometimento da gerência para investir no processo de engenharia de requisitos;
- ceticismo quanto ao valor da engenharia de requisitos;
- relutância em seguir um processo de engenharia de requisitos ou de desenvolvimento mais estruturado;
- políticas e cultura corporativa enraizada;
- conflitos entre *stakeholders*;
- membros da equipe inadequadamente treinados e habilidosos;
- papéis e responsabilidades do projeto não claros;
- falta de propriedade e responsabilidade pelas atividades de requisitos.



Saiba mais

Wiegers e Beatty (2013, p. 561–574) apresentam um conjunto de sintomas que podem ser identificados no desenvolvimento e que são provenientes dos requisitos. Para cada sintoma, a causa provável é apresentada, bem como uma possível solução, de acordo com a categoria do problema: processo, produto, planejamento, comunicação, elicitação, análise, especificação, validação, gestão e gerenciamento de mudanças.

Referências

BOURQUE, P.; FAIRLEY, R. *SWEBOK*: guide to the software engineering body of knowledge: version 3. USA: IEEE Computer Society, 2014. Disponível em: https://www.computer.org/education/bodies-of-knowledge/software-engineering. Acesso em: 29 maio 2020.

CMMI INSTITUTE. *CMMI model V2.0*. Pittsburgh: CMMI Institute, 2018.

CMMI PRDUCT TEAM. *CMMI® for Development, Version 1.3*. Hascom AFB: Software Engineering Institute, 2010. Disponível em: https://resources.sei.cmu.edu/asset_files/TechnicalReport/2010_005_001_15287.pdf. Acesso em: 29 maio 2020.

ISO/IEC/IEEE. *ISO/IEC/IEEE 12207:2017(E)*: systems and software engineering: software life cycle processes. Switzerland: ISO, 2017.

LEFFINGWELL, D. *Agile software requirements*: lean requirements practices for teams, programs, and enterprises. Upper Saddle River: Adisson-Wesley, 2011.

PATTON, J. *User story mapping*: discover the whole story, build the right product. Sebastopol: O´Reilly, 2014.

POHL, K.; RUPP, C. *Requirements engineering fundamentals:* a study guide for the certified professional for requirements engineering exam, foundation level, IREB Compliant. 2. ed. Santa Barbara: Rock Nook, 2015.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMANN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

PREZADO ESTUDANTE

ENCERRA AQUI O TRECHO DO LIVRO DISPONIBILIZADO PELA SAGAH PARA ESTA PARTE DA UNIDADE.

CONTRIBUA COM A QUALIDADE DO SEU CURSO

Se você encontrar algum problema neste material, entre em contato pelo email **eadproducao@unilasalle.edu.br.** Descreva o que você encontrou e indique a página.

Lembre-se: a boa educação se faz com a contribuição de todos!

UNIVERSIDADE
LaSalle
Viver é evoluir.

Av. Victor Barreto, 2288 | Canoas - RS
CEP: 92010-000 | 0800 541 8500
eadproducao@unilasalle.edu.br